UNIVERSIDAD POLITÉCNICA DE VICTORIA

IMPLEMENTACIÓN DE FLUENTES DE ESTADO MULTIVALUADOS EN MDP-PROBLOG MEDIANTE DISYUNCIONES ANOTADAS.

T E S I N A

QUE PARA OBTENER EL GRADO DE

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PRESENTA:

ÁNGEL IVÁN CABRERA ROJAS

DIRECTOR

DR. SAID POLANCO MARTAGÓN

CO-DIRECTOR

DR. HÉCTOR HUGO AVILÉS-ARRIAGA

ORGANISMO RECEPTOR

UNIVERSIDAD POLITÉCNICA DE VICTORIA

CIUDAD VICTORIA, TAMAULIPAS, FEBRERO DE 2026

Ciudad Victoria, Tamaulipas, a **25 de Enero de 2026**

UNIVERSIDAD POLITÉCNICA DE VICTORIA
DR. HÉCTOR HUGO AVILÉS ARRIAGA
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **CABRERA ROJAS ANGEL IVAN** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** y seguro facultativo IMSS número **86-18-04-3681-7;** quién deberá realizar su práctica profesional de **ESTADÍA**, a partir del **07 de Enero de 2026** al **17 de Abril de 2026**, con duración de **600 horas** desarrollando el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** que le fué asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por mail, referente al desempeño del practicante, la información que nos proporcione, es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE

OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN

C.C.P. SAIDPOLANCOMARTAGÓN
ASESOR INSTITUCIONAL

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Victoria Tamaulipas, a 25 de Enero del 2026
Asunto: Carta de Aceptación

M.A OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN
UNIVERSIDAD POLITÉCNICA DE VICTORIA
PRESENTE

Hacemos de su conocimiento que hemos aceptado al estudiante **CABRERA ROJAS ANGEL IVAN** del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** de la Universidad Politécnica de Victoria, para realizar su **ESTADÍA** en nuestra empresa **UNIVERSIDAD POLITÉCNICA DE VICTORIA**, durante el periodo comprendido del día **07 de Enero de 2026** al **17 de Abril de 2026**, el estudiante estará colaborando en el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** con una carga horaria total de **600 horas**

Al concluir satisfactoriamente sus prácticas profesionales, se le entregará al estudiante su carta de liberación debidamente formalizada.

Con el objetivo de colaborar en la mejora de los programas académicos de la Universidad Politécnica de Victoria, estamos de acuerdo en responder a la brevedad posible el formulario de evaluación del desempeño del practicante previo a emitir la liberación de **ESTADÍA**.

Sin otro particular.

ATENTAMENTE
DR. HÉCTOR HUGO AVILÉS ARRIAGA
ASESOR EMPRESARIAL

C.C.P. ARCHIVO

Ing. Juan Camaney de Alba Rojas
PRESENTE

INSERTAR AQUÍ
DOCUMENDO
DE CARTA
DE LIBERACION
DEBIDAMENTE
FIRMADO

CARTA DE ACEPTACIÓN DEL DOCUMENTO PARA SU IMPRESIÓN

Cd. Victoria, Tamaulipas a x

Ángel Iván Cabrera Rojas
PRESENTE

Le comunico que el Programa Académico de Ingeniería en Tecnologías de la Información le ha otorgado la autorización para la impresión de su Tesina de Estadía Práctica cuyo título es:

Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.

ATENTAMENTE

Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

c.c.p Director de programa académico

EVALUACIÓN DE ESTADÍA
Rúbrica para evaluación de la presentación y el reporte de estadía

Nombre del alumno: **ÁNGEL IVÁN CABRERA ROJAS** Calificación final: _____

Periodo: **X**

Ponderación	Aspecto a Evaluar	Competente 10	Independiente 9	Básico Avanzado 8	No Competente 5
40	Resultados y Actividades	Estrechamente relacionados al perfil de egreso de su programa académico	Parcialmente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico
30	Exposición de las actividades de la estadía	Detalladas y sustentadas con respecto a los resultados que se obtuvieron	Detalladas y sustentadas parcialmente con respecto a los resultados que se obtuvieron	Detalladas parcialmente con respecto a los resultados que se obtuvieron	Detalladas escazamente con respecto a los resultados que se obtuvieron
10	Material visual Lenguaje verbal	Uso el lenguaje y la terminología apropiadas; El material visual está organizado, adecuado y suficiente	Uso el lenguaje y la terminología apropiadas El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y la terminología son parcialmente apropiadas; El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y terminología es inapropiado; El material visual no está organizado y es insuficiente
10	Exposición en Idioma Inglés	Pronunciation is clear so language is easily understood (2.5) Uses fluent connected speech, occasionally disrupted by search for correct form of expression (2.5) Uses topic related vocabulary without problems (2.5) Responds to questions using varied and descriptive vocabulary and language structures (2.5)	Pronunciation is understandable, but there are slight errors (2.25) Speech is connected but frequently disrupted by search for correct form of expression (2.25) Uses some topic related vocabulary sufficient to communicate ideas (2.25) Responds to questions using simple but accurate vocabulary and language structures (2.25	Pronunciation is understandable most of the time, marked native accent and many errors (2) Speaks with simple sentences, sometimes not connected, but is understood (2) Uses basic vocabulary to communicate ideas (2) Partly responds to simple questions, with limited vocabulary and language structures (2)	Pronunciation makes language very difficult to understand (1) Uses one-word/two-word utterances (1) Unable to communicate ideas due to lack of vocabulary (1) Uses isolated words or sentence fragments to respond to questions (1)
5	Respuesta a los cuestiona-mientos de los evaluadores	Clara y satisfactoria	Clara y parcialmente satisfactoria	Clara e insuficiente	Confusa e insuficiente
5	Autorización de tesina en tiempo y forma	Presenta en tiempo y forma	Presenta en tiempo y forma con la mayoría de requerimientos solicitados	Presenta en tiempo y con algunas limitantes de los requerimientos solicitados.	Presenta fuera de tiempo y con los mínimos requerimientos solicitados.

Dr. Said Polanco Martagón	X	
ASESOR INSTITUCIONAL	EVALUADOR	EVALUADOR DE INGLÉS

REGISTRO DE EVALUACIÓN DE EXPOSICIÓN DE ESTADÍA

Siendo las X horas del día X, el alumno **Ángel Iván Cabrera Rojas**, del programa académico **Ingeniería en Tecnologías de la Información**, con matricula **2230343**, presentó la exposición de la estadía realizada durante el cuatrimestre **x**, en la **Universidad Politécnica de Victoria**, con el proyecto titulado **Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.**.

Una vez concluido el proceso de evaluación, y con base a la rúbrica establecida para éste propósito, se determina que la calificación de la estadía es _____.

_____
Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

_____
X
EVALUADOR

_____

EVALUADOR DE INGLÉS

Agradecimientos

Resumen

La Programación Lógica Probabilística (PLP) es un paradigma que combina la capacidad de representación de la lógica de primer orden con la teoría de la probabilidad para el manejo de la incertidumbre. Este paradigma permite modelar una amplia variedad de dominios que abarcan situaciones de la vida real en donde existen interacciones complejas entre conjuntos de elementos heterogéneos y distintos niveles de incertidumbre. Dentro de este contexto, destaca el framework MDP-ProbLog, el cual utiliza las capacidades del lenguaje ProbLog para la especificación y resolución de Procesos de Decisión de Markov (MDPs) de horizonte infinito utilizando una representación factorizada, lógica y relacional.

Sin embargo, la arquitectura base de este sistema presenta una limitación estructural: la representación de los fluentes de estado se restringe exclusivamente a variables de Bernoulli independientes (booleanas). Debido a esto, para modelar variables que por naturaleza son multivaluadas, el usuario se ve obligado a utilizar técnicas para descomponerlas en múltiples predicados binarios. En el caso de MDP-ProbLog, esta restricción resulta en un proceso de modelado tedioso, propenso a inconsistencias semánticas y poco natural.

La presente tesina reporta el desarrollo de una extensión al núcleo del framework MDP-ProbLog para solucionar esta problemática. El trabajo abarca la revisión de la arquitectura del código fuente, el planteamiento y diseño de la extensión, la programación, la ejecución de pruebas y su respectiva documentación. Mediante modificaciones a los módulos de gestión de estados y del motor de inferencia, se integró el soporte para fluentes de estado multivaluados utilizando el mecanismo nativo de las Disyunciones Anotadas (Annotated Disjunctions). Como resultado de este desarrollo, el modelado de MDPs que requieren variables con dominios multivaluados se convierte en un proceso semánticamente íntegro, mucho más directo y natural para el usuario.

Palabras clave: Programación Lógica Probabilística, MDP-ProbLog, Disyunciones Anotadas, Fluentes Multivaluados, Espacio de Estados.

Summary

Probabilistic Logic Programming (PLP) is a paradigm that combines the representational power of first-order logic with probability theory to handle uncertainty. This paradigm allows for the modeling of a wide variety of domains encompassing real-world situations where complex interactions exist among sets of heterogeneous elements and varying levels of uncertainty. Within this context, the MDP-ProbLog framework stands out, leveraging the capabilities of the ProbLog language for the specification and resolution of infinite-horizon Markov Decision Processes (MDPs) using a factored, logical, and relational representation.

However, the base architecture of this system presents a structural limitation: the representation of state fluents is restricted exclusively to independent Bernoulli (Boolean) variables. Due to this, to model variables that are naturally multi-valued, the user is forced to use techniques to decompose them into multiple binary predicates. In the case of MDP-ProbLog, this restriction results in a modeling process that is tedious, prone to semantic inconsistencies, and unnatural.

This thesis reports the development of an extension to the core of the MDP-ProbLog framework to address this problem. The work encompasses the review of the source code architecture, the planning and design of the extension, programming, testing, and its respective documentation. Through modifications to the state management modules and the inference engine, support for multi-valued state fluents was integrated using the native mechanism of Annotated Disjunctions. As a result of this development, the modeling of MDPs requiring variables with multi-valued domains becomes a semantically sound, much more direct, and natural process for the user.

Keywords: Probabilistic Logic Programming, MDP-ProbLog, Annotated Disjunctions, Multi-valued Fluents, State Space.

Índice

Agradecimientos vii

Resumen viii

Summary ix

Índice x

1. Introducción 1
- 1.1. Antecedentes locales . . . 2
- 1.2. Definición del Problema . . . 3
- 1.3. Objetivos . . . 3
  - 1.3.1. Objetivo General . . . 3
  - 1.3.2. Objetivos Específicos . . . 3
- 1.4. Justificación . . . 4
- 1.5. Alcances y Limitaciones . . . 5
  - 1.5.1. Alcances . . . 5
  - 1.5.2. Limitaciones . . . 6

2. Estado del Arte 7
- 2.1. MDPs Factorizados y el Problema de la Representación de Estados . . . 7
- 2.2. Programación Lógica Probabilística para la Resolución de MDPs . . . 8
- 2.3. Disyunciones Anotadas y Compilación de Conocimiento . . . 9
- 2.4. Análisis Comparativo . . . 10
- 2.5. Conclusión del Estado del Arte . . . 11

3. Marco Teórico 12
- 3.1. Programación Lógica . . . 12
- 3.2. Programación Lógica Probabilística . . . 12
  - 3.2.1. Semántica de distribución . . . 12
  - 3.2.2. El lenguaje ProbLog . . . 14
  - 3.2.3. Relevancia para MDP-ProbLog . . . 15
- 3.3. Conteo de Modelos Ponderados . . . 15
- 3.4. Procesos de Decisión de Markov . . . 16
  - 3.4.1. Algoritmo de Iteración de Valor . . . 16
- 3.5. MDPs Factorizados . . . 17
  - 3.5.1. Fluentes de estado . . . 18
- 3.6. El Framework MDP-ProbLog . . . 18
- 3.7. Disyunciones Anotadas . . . 18
  - 3.7.1. Definición formal de LPADs . . . 18
  - 3.7.2. Semántica: selecciones e instancias . . . 19
  - 3.7.3. Exclusión mutua nativa . . . 20
  - 3.7.4. Interpretación causal . . . 21
- 3.8. Compilación de Conocimiento . . . 21

3.8.1. Mapa de compilación de conocimiento 21
3.8.2. Impacto de la representación en el tamaño del circuito 22

4. Diseño e Implementación **23**
4.1. Análisis de la arquitectura original 23
4.1.1. Componentes del sistema original 23
4.1.2. Flujo de ejecución 24
4.1.3. Limitaciones en la gestión de fluentes 24
4.1.4. Identificación de puntos de extensión 25
4.2. Diseño de la extensión 25
4.2.1. FluentSchema: estructura factorizada del espacio de estados 25
4.2.2. FluentClassifier: mecanismo de clasificación automática 26
4.2.3. Codificación *mixed-radix* del espacio de estados 27
4.2.4. Sintaxis de usuario para fluentes multivaluados 28
4.3. Implementación 30
4.3.1. Adaptador al motor de inferencia 30
4.3.2. Coordinador central del MDP 30
4.3.3. Adaptaciones al algoritmo de resolución 31
4.3.4. Herramientas de verificación y exportación 32
4.4. Retrocompatibilidad 32

5. Experimentación y Resultados **34**
5.1. Diseño experimental 34
5.1.1. Dominio de prueba 34
5.1.2. Codificaciones comparadas 35
5.1.3. Métricas de evaluación 37
5.1.4. Protocolo de medición 37
5.1.5. Entorno de pruebas 38
5.2. Verificación de corrección funcional 38
5.3. Simplificación del proceso de modelado 39
5.4. Impacto computacional 40
5.4.1. Tamaño del circuito compilado 40
5.4.2. Tiempos de ejecución 40
5.5. Discusión 41
5.5.1. Corrección y convergencia 41
5.5.2. Simplificación del modelado 41
5.5.3. Comportamiento computacional 42
5.5.4. Relación con las predicciones del Marco Teórico 42
5.5.5. Limitaciones de la evaluación 42

Índice de figuras **43**

Índice de cuadros **44**

Índice de algoritmos **45**

Referencias **46**

1. Introducción

Dentro del ámbito de la Inteligencia Artificial, la Programación Lógica Probabilística se ha consolidado como uno de los paradigmas fundacionales el cual combina la expresividad de la lógica de primer orden con el manejo formal de la incertidumbre mediante la teoría de la probabilidad [1]. Dentro de los desarrollos apegados a este paradigma se encuentra *ProbLog*, un lenguaje que extiende Prolog mediante la introducción de hechos probabilísticos y cuya semántica se basa en la distribución sobre los mundos posibles [2]. ProbLog permite calcular la probabilidad de que una consulta lógica sea verdadera dado un conjunto de evidencias, utilizando técnicas de inferencia como el Conteo de Modelos Ponderados (Weighted Model Counting), transformando el problema de inferencia en una evaluación eficiente sobre circuitos lógicos compilados [3].

Paralelamente, en el campo de la toma de decisiones secuenciales, los Procesos de Decisión de Markov se han establecido como el estándar matemático para modelar agentes que actúan en entornos estocásticos. Formalizados rigurosamente a mediados del siglo XX con los trabajos de Richard Bellman sobre programación dinámica [4], los MDPs proporcionan un marco formal para optimizar una función de utilidad a lo largo del tiempo. Sin embargo, su representación clásica mediante matrices de estados atómicos presenta serias limitaciones de escalabilidad. A medida que aumenta el número de variables que describen el estado del sistema, el tamaño del espacio de estados crece de manera exponencial, un fenómeno acuñado por Bellman como la "maldición de la dimensionalidad" [4]. Esta explosión combinatoria hace que los métodos de resolución clásicos sean computacionalmente intratables para problemas del mundo real con múltiples variables interrelacionadas [5].

Integrando la expresividad lógica de ProbLog y la capacidad de decisión de los MDPs, Bueno et al. propusieron el framework **MDP-ProbLog** [6]. Esta herramienta integra los conceptos de estados, acciones y recompensas dentro de la sintaxis de la programación lógica, permitiendo representar MDPs de horizonte infinito de manera relacional y compacta. Al utilizar una representación factorizada, MDP-ProbLog mitiga parcialmente la problemática de la dimensionalidad al descomponer el estado global en un conjunto de características o fluentes. Actualmente, este programa es la base tecnológica del equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga, quienes lo utilizan para el desarrollo de sistemas de navegación y toma de decisiones en vehículos autónomos.

1.1. Antecedentes locales

Actualmente, MDP-ProbLog constituye la base tecnológica del equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga, quienes lo utilizan para el desarrollo de sistemas de navegación y toma de decisiones en vehículos autónomos. Sin embargo, el framework conserva desde su publicación una restricción que limita la naturalidad del modelado. Todos los fluentes de estado se definen exclusivamente como variables de Bernoulli independientes, lo que obliga al usuario a representar variables multivaluadas mediante múltiples predicados booleanos. Para modelar la posición de un vehículo en una vía de tres carriles, por ejemplo, se requieren tres fluentes binarios separados en lugar de una única variable con tres valores. Esta codificación forzada produce un proceso de modelado tedioso, propenso a inconsistencias semánticas, y genera un espacio de estados artificialmente inflado.

El presente trabajo aborda el diseño e implementación de fluentes de estado multivaluados en el núcleo de MDP-ProbLog mediante el uso de Disyunciones Anotadas (ADs) [7], con el objetivo de extender las capacidades de representación del sistema, garantizando la integridad semántica de los modelos y sentando las bases para analizar el impacto computacional de esta expresividad.

1.2. Definición del Problema

La arquitectura actual de MDP-ProbLog carece de un mecanismo nativo para representar y evaluar variables de estado que posean más de dos valores mutuamente excluyentes. Esta restricción a fluentes booleanos independientes genera una desconexión fundamental entre la naturaleza multivaluada de los fenómenos físicos del mundo real y su representación computacional.

Esta limitación delega en el usuario la responsabilidad de mantener la consistencia semántica mediante la escritura exhaustiva de reglas lógicas adicionales. La ausencia de soporte para fluentes multivaluados resulta en un proceso de modelado tedioso y poco natural, aumentando la propensión a errores humanos en la definición de las políticas de decisión y generando un espacio de estados artificialmente inflado con combinaciones semánticamente imposibles.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e integrar un mecanismo de representación de fluentes de estado multivaluados en el núcleo del framework MDP-ProbLog utilizando Disyunciones Anotadas (Annotated Disjunctions), con el fin de proveer una sintaxis de modelado más natural, asegurar la consistencia semántica en los procesos de decisión estocásticos y evaluar el impacto computacional derivado de esta nueva expresividad.

1.3.2. Objetivos Específicos

- Analizar la arquitectura interna actual de MDP-ProbLog, identificando los módulos críticos encargados de la gestión del espacio de estados y la inferencia probabilística para determinar los puntos de extensión.
- Desarrollar una estructura de datos y adaptar la lógica de instanciación (*grounding*) que permita procesar fluentes multivaluados frente a los fluentes binarios tradicionales.
- Implementar el soporte para Disyunciones Anotadas en la definición de las transiciones de estado, garantizando matemáticamente la exclusión mutua nativa de los valores de una variable sin necesidad de reglas de restricción externas.
- Evaluar el comportamiento de la extensión propuesta, contrastando la simplificación cualitativa del proceso de modelado con las implicaciones cuantitativas generadas en el tamaño de los modelos compilados y los tiempos de inferencia.

1.4. Justificación

El trabajo abarca un paso necesario para la maduración de la herramienta. Desde una perspectiva práctica y operativa, la integración de fluentes multivaluados impacta directamente la manera en la que se modelan y describen los MDPs. Al automatizar la exclusión mutua mediante Disyunciones Anotadas, se elimina la necesidad de crear reglas manuales restrictivas, reduciendo drásticamente la carga cognitiva del modelador, acortando los tiempos de desarrollo y mitigando la propensión a errores de diseño.

Para dimensionar el problema: una variable con k valores posibles requiere, bajo la codificación booleana actual, k predicados independientes que generan 2^k combinaciones, de las cuales solo k son semánticamente válidas. Por ejemplo, un semáforo de tres colores produce $2^3 = 8$ estados de los cuales 5 son imposibles por definicion; un fluente con 5 valores genera $2^5 = 32$ combinaciones en donde únicamente 5 son válidas. Con la extensión propuesta, la representación se reduce exactamente a los k estados legítimos.

[Diagrama o figura en donde se muestre la diferencia entre la representación actual y la propuesta, destacando la reducción del espacio de estados y la simplificación del modelado.]

1.5. Alcances y Limitaciones

1.5.1. Alcances

El proyecto abarca el diseño, la implementación y la validación de una extensión al núcleo del framework MDP-ProbLog que incorpora soporte para fluentes de estado multivaluados. El desarrollo comprende los siguientes elementos:

1. **Representación factorizada del espacio de estados.** Se diseñaron las estructuras de datos necesarias para que el espacio de estados admita factores con más de dos valores posibles, preservando la capacidad de iteración y la indexación eficiente sobre el conjunto completo de estados.

2. **Clasificación automática de fluentes.** Se implementó un mecanismo que determina automáticamente si un fluente declarado por el usuario es booleano o multivaluado, a partir del análisis de las Disyunciones Anotadas presentes en el programa. El usuario también puede especificar el tipo de forma explícita.

3. **Integración con el motor de inferencia.** Se adaptó la interfaz entre MDP-ProbLog y el motor de inferencia de ProbLog para soportar la inyección y el manejo de Disyunciones Anotadas como representación nativa de los fluentes multivaluados.

4. **Adaptación del algoritmo de resolución.** Se reestructuró el flujo de evaluación de transiciones y el algoritmo de Iteración de Valor para operar sobre transiciones agrupadas por factor, de modo que cada variable de estado se evalúa de acuerdo con su cardinalidad real.

5. **Herramientas de inspección.** Se desarrollaron utilidades para exportar las matrices de transición, las tablas de recompensas y el historial de convergencia, facilitando la verificación y el análisis de los modelos resueltos.

6. **Retrocompatibilidad.** La extensión preserva el funcionamiento de los modelos booleanos existentes sin requerir modificaciones por parte del usuario.

La validación funcional se realiza sobre dominios de referencia basados en grids, específicamente la grid de Mitchell y la grid de Russell, que constituyen problemas canónicos en la literatura de MDPs con soluciones óptimas conocidas. En cada dominio se contrastan dos formulaciones del mismo problema, una con codificación booleana manual y otra con fluentes multivaluados, evaluando la simplificación del proceso de modelado, la corrección de la política resultante y el comportamiento del solver en términos de convergencia y tamaño del espacio de estados.

1.5.2. Limitaciones

El desarrollo se circunscribe a MDPs de horizonte infinito con espacios de estados discretos y completamente observables. Las siguientes restricciones delimitan el alcance técnico de la extensión:

1. **Enumeración explícita del espacio de estados.** El algoritmo de Iteración de Valor recorre todos los pares estado-acción en cada iteración. La extensión modifica la forma en que se representan y se evalúan los estados, pero no altera la estrategia de recorrido. El costo por iteración permanece proporcional al producto del número total de estados por el número de acciones disponibles. Esta restricción no es introducida por el presente trabajo, sino heredada del diseño original de MDP-ProbLog, y constituye la frontera principal de escalabilidad del sistema. Dominios con un número elevado de variables de estado o con variables de cardinalidad alta producen espacios de estados cuya enumeración completa se vuelve computacionalmente costosa.

2. **Capa de representación exclusivamente.** El trabajo no modifica los algoritmos de inferencia exacta ni los compiladores de circuitos lógicos utilizados internamente por ProbLog. El efecto de las Disyunciones Anotadas sobre el tamaño de los circuitos compilados es un resultado emergente del compilador subyacente, no una optimización introducida por esta extensión.

3. **Convención de declaración de fluentes.** Todos los valores posibles de un fluente multivaluado deben declararse bajo el mismo nombre de predicado. Esta convención permite que el sistema identifique automáticamente los grupos mutuamente excluyentes, pero impide representar un fluente cuyos valores se distribuyan entre predicados distintos.

4. **Independencia entre variables de estado.** Las reglas de transición del programa deben respetar independencia entre las variables de estado. El sistema evalúa la probabilidad de transición de cada variable por separado y combina los resultados asumiendo que los factores son estocásticamente independientes. Si las reglas del usuario introducen dependencias entre variables que violan este supuesto, el sistema no lo detecta ni emite advertencias.

5. **Dominios de evaluación.** Las pruebas se ejecutan sobre dominios de grid discretos de dimensión reducida, seleccionados por la disponibilidad de soluciones de referencia en la literatura. Los resultados no se extrapolan a dominios de gran escala ni a entornos físicos en tiempo real.

Una vez delimitado el problema, los objetivos y los alcances de este trabajo, el siguiente capítulo analiza los antecedentes científicos que sustentan la propuesta, revisando los enfoques existentes para la representación factorizada de MDPs y el papel de la Programación Lógica Probabilística en la toma de decisiones secuenciales.

2. Estado del Arte

El presente capítulo revisa la literatura científica relacionada con la sustentación técnica de esta tesina: la representación factorizada de MDPs y el problema de las variables de estado, el uso de la Programación Lógica Probabilística como lenguaje de especificación para la toma de decisiones secuenciales, y los mecanismos de exclusión mutua y compilación de conocimiento que permiten la inferencia eficiente. El análisis de estos trabajos permite trazar la trayectoria que conduce al vacío de investigación que la presente tesina aborda: la ausencia de soporte nativo para fluentes de estado multivaluados en frameworks de resolución exacta de MDPs basados en PLP.

2.1. MDPs Factorizados y el Problema de la Representación de Estados

En la formulación clásica de un MDP, el espacio de estados se trata como un conjunto plano donde cada estado es una entidad atómica e indivisible [5]. Esta representación se vuelve poco práctica conforme aumenta el número de variables descritas: si un dominio involucra n variables binarias, el número total de estados crece como 2^n, lo que Bellman denominó la *maldición de la dimensionalidad* [4].

Para mitigar este crecimiento exponencial, Boutilier, Dearden y Goldszmidt propusieron la representación factorizada de MDPs, donde las transiciones se codifican mediante Redes Bayesianas Dinámicas (DBNs) y las funciones de valor se manipulan como árboles de decisión [8]. En esta representación, cada variable de estado posee su propia tabla de probabilidad condicional, y los algoritmos de programación dinámica operan sobre agregaciones de estados en lugar de iterar sobre cada estado individual. Su algoritmo *Structured Policy Iteration* (SPI) demostró que era posible resolver MDPs sin enumerar explícitamente el espacio de estados, siempre que la estructura del problema exhibiera independencias condicionales aprovechables.

En la misma línea, Hoey *et al.* desarrollaron SPUDD, un algoritmo de iteración de valor que reemplaza los árboles de decisión por Diagramas de Decisión Algebraicos (ADDs) [9]. Los ADDs, al ser grafos dirigidos acíclicos en lugar de árboles, permiten compartir subgrafos isomorfos, logrando reducciones de hasta 30 veces en el número de nodos respecto a las representaciones arbóreas. SPUDD demostró la resolución exacta de MDPs con hasta 63 millones de estados.

Tanto SPI como SPUDD comparten una restricción, ya que operan exclusivamente sobre variables booleanas. Cuando el dominio contiene variables con más de dos valores, el modelador debe descomponer cada variable multivaluada en múltiples variables binarias. Hoey *et al.* reconocen explícitamente esta limitación al señalar que variables ternarias deben expandirse en dos binarias, lo cual agranda el espacio de estados por un factor de 4/3 por cada variable expandida e introduce estados semánticamente imposibles [9]. Este problema de la binarización forzada es precisamente la limitación que la presente tesina aborda en el contexto de MDP-ProbLog.

En el ámbito de los lenguajes de descripción para planificación, RDDL (*Relational Dynamic Influence Diagram Language*) [10] sí contempla fluentes multivaluados de manera nativa, incluyendo variables enumeradas, enteras y continuas. No obstante, RDDL está fundamentado en redes bayesianas dinámicas proposicionales, lo que impide la representación de dependencias cíclicas que surgen naturalmente en dominios relacionales. MDP-ProbLog, en contraste, hereda de la programación lógica la capacidad de expresar este tipo de relaciones, pero carece del soporte multivaluado que RDDL ofrece.

2.2. Programación Lógica Probabilística para la Resolución de MDPs

La Programación Lógica Probabilística (PLP) combina la capacidad de representación relacional de la lógica de primer orden con el manejo formal de la incertidumbre mediante la teoría de la probabilidad. De Raedt y Kimmig establecieron los fundamentos unificadores de este paradigma, demostrando cómo la *semántica de distribución* permite definir una distribución de probabilidad sobre los mundos posibles de un programa lógico [11]. Bajo esta semántica, cada hecho probabilístico actúa como una variable aleatoria independiente cuya realización determina un programa lógico determinista; la probabilidad de una consulta es entonces la suma de las probabilidades de todos los mundos en los que dicha consulta es verdadera.

ProbLog, el lenguaje que implementa esta semántica, transforma el problema de inferencia probabilística en un problema de Conteo de Modelos Ponderados (WMC) sobre fórmulas booleanas [3, 1]. Esto permite aprovechar técnicas de compilación de conocimiento para lograr inferencia exacta en tiempo polinomial respecto al tamaño del circuito compilado.

La extensión de ProbLog hacia la toma de decisiones se materializó primero en DTProbLog [12], que añadió al lenguaje la capacidad de modelar problemas de decisión episódicos: dado un conjunto de variables de decisión y atributos de utilidad, el sistema encuentra la estrategia que maximiza la utilidad esperada. Sin embargo, DTProbLog está diseñado para problemas de un solo paso y no soporta la resolución de MDPs de horizonte infinito, donde es necesario optimizar una política completa a lo largo del tiempo.

Como derivado de este trabajo, Bueno *et al.* propusieron MDP-ProbLog, el cual extiende la sintaxis de ProbLog con predicados reservados para declarar fluentes de estado, acciones y utilidades, y emplea programas de distribución de transición de dos pasos temporales para representar la dinámica del MDP [6]. El *solver* implementa iteración de valor sobre el circuito compilado mediante WMC, demostrando formalmente la equivalencia entre ejecutar el programa probabilístico y realizar un *backup* de Bellman. Una ventaja distintiva de MDP-ProbLog frente a lenguajes como RDDL es su capacidad de representar dependencias cíclicas entre predicados aterrizados gracias a la resolución SLD de Prolog [6].

2.3. Disyunciones Anotadas y Compilación de Conocimiento

El mecanismo que esta tesina propone para resolver la restricción binaria de MDP-ProbLog se fundamenta en las Disyunciones Anotadas (ADs). En la literatura de representación del conocimiento, Vennekens, Verbaeten y Bruynooghe introdujeron los Logic Programs with Annotated Disjunctions (LPADs) como una solución elegante para modelar eventos probabilísticos con múltiples resultados mutuamente excluyentes. El principal resultado de su trabajo demostró que es posible codificar la exclusión mutua de manera nativa en la cabeza de las reglas lógicas, eliminando la necesidad de escribir restricciones manuales complejas y mitigando la generación de estados semánticamente imposibles.

Adicionalmente, Vennekens et al. demostraron empírica y formalmente que las ADs admiten una interpretación causal natural para problemas de toma de decisiones. Esta característica es crucial, ya que permite mapear de forma directa la semántica del lenguaje a las transiciones estocásticas de un MDP: dado un estado y una acción (la causa), el sistema transita hacia uno de múltiples estados posibles (los efectos). En esta misma línea, demostraron la equivalencia expresiva entre su propuesta y el Independent Choice Logic (ICL) de Poole, consolidando a las ADs como un estándar más natural para representar efectos indeterminados.

Ahora bien, para que la inferencia probabilística sobre programas con ADs sea computacionalmente tratable en la práctica, sistemas como ProbLog emplean un flujo de compilación de conocimiento. El programa lógico aterrizado se transforma primero en una fórmula proposicional ponderada, y posteriormente se compila en un circuito lógico. En este ámbito, Darwiche y Marquis establecieron la taxonomía de los lenguajes de compilación, identificando que estructuras como las d-DNNFs (Deterministic Decomposable Negation Normal Forms) permiten realizar inferencia exacta en tiempo lineal respecto al tamaño del circuito compilado [13]. Posteriormente, el mismo autor introdujo los Diagramas de Decisión Sentencial (SDDs), una representación canónica que generaliza los clásicos BDDs y resulta estrictamente más sucinta [14]. En la práctica, el motor subyacente de ProbLog2, brinda la flexibilidad de compilar el modelo hacia cualquiera de estas dos estructuras [15]. Es precisamente sobre estos circuitos donde el sistema ejecuta el Conteo de Modelos Ponderados (WMC) para calcular las probabilidades de transición. Por lo tanto, cualquier extensión a la capa de representación, como la integración de fluentes multivaluados propuesta en este trabajo, debe garantizar su compatibilidad con esta forma de compilación para preservar la eficiencia computacional.

2.4. Análisis Comparativo

La Tabla 1 presenta una comparativa de los principales enfoques revisados en este capítulo. Se analizan diez trabajos representativos agrupados por familia metodológica. Para cada uno se evalúan cuatro dimensiones: el tipo de variables de estado que soporta, el mecanismo de inferencia empleado, la capacidad del lenguaje para expresar relaciones entre entidades, y la forma en que se garantiza la exclusión mutua entre los valores de una misma variable.

Tabla 1: Análisis comparativo del estado del arte. Se evalúan cuatro dimensiones para cada trabajo: el tipo de variables de estado soportadas, el mecanismo de inferencia, la capacidad relacional del lenguaje, y la garantía de exclusión mutua entre valores de una misma variable.

Trabajo	**Variables de estado**	**Inferencia**	**Expresividad relacional**	**Exclusión mutua**
MDPs factorizados proposicionales				
Boutilier *et al.* [8]	Booleanas	Exacta (árboles)	Proposicional (DBNs)	Manual
Boutilier, Dean y Hanks [16]	Booleanas	Exacta (prog. dinámica)	Proposicional	Manual
Hoey *et al.* [9]	Booleanas (binarización)	Exacta (ADDs)	Proposicional (DBNs)	Manual
Lenguajes de planificación				
Sanner [10]	Multival., continuas	Exacta / Aprox.	Relacional (DBNs)	Nativa por tipo
Programación Lógica Probabilística				
De Raedt y Kimmig [11]	Booleanas	Exacta (WMC)	Relacional (ProbLog)	No aplica
Fierens *et al.* [3]	Booleanas	Exacta (WMC)	Relacional (ProbLog)	No aplica
Nitti *et al.* [17]	Discretas, continuas	Aprox. (muestreo)	Relacional (DDC)	Implícita
Riguzzi y Swift [18]	Booleanas	Exacta (tabling)	Relacional	No aplica
Exclusión mutua y semántica causal				
Vennekens *et al.* [19]	Multivaluadas (ADs)	Exacta	Relacional	Nativa
Poole [20]	Booleanas (ICL)	Exacta	Relacional	Nativa (elecciones)
MDP-ProbLog				
Bueno *et al.* [6]	Booleanas	Exacta (WMC/SDDs)	Relacional (ProbLog)	Manual
Propuesta	**Multivaluadas**	**Exacta (WMC/SDDs)**	**Relacional (ProbLog)**	**Nativa (ADs)**

Como se observa en la tabla, ninguno de los enfoques existentes integra simultáneamente las cuatro capacidades que la propuesta de esta tesina reúne: variables de estado multivaluadas con exclusión mutua nativa, inferencia exacta mediante WMC sobre circuitos compilados, y expresividad relacional completa heredada de la programación lógica.

2.5. Conclusión del Estado del Arte

La revisión de la literatura permite identificar tres hallazgos principales. En primer lugar, la representación factorizada de MDPs ha demostrado ser una estrategia efectiva para mitigar la maldición de la dimensionalidad, permitiendo resolver problemas con millones de estados sin enumeración explícita [8, 9]. Sin embargo, los enfoques proposicionales más consolidados operan exclusivamente sobre variables booleanas, y cuando el dominio requiere variables multivaluadas, la binarización forzada introduce estados semánticamente imposibles y aumenta artificialmente el tamaño de los circuitos compilados.

En segundo lugar, la Programación Lógica Probabilística ofrece ventajas significativas sobre los formalismos proposicionales al permitir representar relaciones simétricas, transitivas y cíclicas entre las variables del dominio [6, 11]. El framework MDP-ProbLog materializó esta ventaja al combinar la expresividad de ProbLog con la resolución exacta de MDPs de horizonte infinito mediante WMC [3]. No obstante, su arquitectura restringe los fluentes de estado a variables de Bernoulli independientes, lo que limita la naturalidad del proceso de modelado y delega en el usuario la responsabilidad de codificar manualmente las restricciones de exclusión mutua.

En tercer lugar, las Disyunciones Anotadas constituyen un mecanismo formal que garantiza la exclusión mutua de eventos probabilísticos a nivel semántico [19, 20]. Este constructo ya forma parte del lenguaje ProbLog y es utilizado internamente por su motor de inferencia [15], pero su integración en la capa de representación de fluentes de estado dentro de MDP-ProbLog no ha sido explorada ni implementada.

En consecuencia, la presente tesina se posiciona en la intersección de estos tres ejes al proponer la extensión de MDP-ProbLog para soportar fluentes de estado multivaluados mediante Disyunciones Anotadas. Esta integración busca preservar las garantías de inferencia exacta del sistema, eliminar la necesidad de binarización manual, y habilitar una representación del espacio de estados que sea tanto más compacta como semánticamente íntegra. El siguiente capítulo establece los fundamentos formales — desde la programación lógica hasta la compilación de conocimiento — que el lector necesitará para comprender el diseño de la extensión propuesta.

3. Marco Teórico

Una vez identificada la brecha en la literatura, el presente capítulo establece los fundamentos teóricos y formales que el lector necesitará para comprender el diseño de la extensión propuesta. La exposición sigue un orden ascendente de abstracción: se parte de los conceptos básicos de la programación lógica (§3.1), se incorpora la incertidumbre mediante la semántica de distribución y el lenguaje ProbLog (§3.2), se introduce el Conteo de Modelos Ponderados como mecanismo de inferencia (§3.3), se formalizan los MDPs y su representación factorizada (§3.4–§3.5), se describe la mecánica operativa de MDP-ProbLog (§3.6), y se concluye con las Disyunciones Anotadas (§3.7) y la compilación de conocimiento (§3.8), los dos pilares formales sobre los que se construye la extensión.

3.1. Programación Lógica

Dado que MDP-ProbLog está construido sobre el lenguaje Prolog, es necesario establecer los fundamentos de la programación lógica que sustentan tanto la representación del conocimiento como el proceso de inferencia del sistema. La programación lógica es un paradigma donde los programas se expresan como conjuntos de sentencias lógicas y la ejecución consiste en realizar inferencias sobre ellas. El vocabulario se construye a partir de *constantes* (valores concretos como `rojo`), *variables* (denotadas con mayúscula inicial, como `X`) y *functores* (nombres de predicados con aridad asociada). Un *término* es una constante, variable o functor aplicado a otros términos; un *átomo* es un predicado aplicado a sus argumentos (e.g., `color(carro, rojo)`); y un *literal* es un átomo o su negación. A partir de estos elementos se construyen las cláusulas del programa, las cuales pueden ser *hechos* (afirmaciones incondicionales que declaran verdades absolutas, e.g., `padre(tom, bob).`), *reglas* (implicaciones condicionales donde la cabeza es verdadera si el cuerpo se satisface, e.g., `abuelo(X,Z) :- padre(X,Y), padre(Y,Z).`), y *consultas*, las cuales consisten en preguntas al sistema sobre qué es inferible.

3.2. Programación Lógica Probabilística

La semántica de distribución define el marco probabilístico bajo el cual opera el motor de inferencia de MDP-ProbLog y constituye la base formal para el cálculo exacto de las probabilidades de transición entre estados. La Programación Lógica Probabilística (PLP) extiende la programación lógica clásica con la capacidad de manejar incertidumbre de manera formal. Mientras que un programa lógico clásico afirma que ciertos hechos son verdaderos de manera determinista, un programa lógico probabilístico permite que los hechos sean verdaderos con una cierta probabilidad.

3.2.1. Semántica de distribución

La base semántica de la PLP fue establecida por Sato mediante la *semántica de distribución* (*distribution semantics*) [2]. Su formulación parte de una observación elegante: si se asigna una distribución de probabilidad a los hechos de un programa lógico, las reglas del programa propagan esa incertidumbre de manera determinista, induciendo una distribución sobre el conjunto de todos los modelos mínimos posibles.

Formalmente, considérese un programa $DB = F \cup R$, donde $F = \{f_1, f_2, \ldots, f_k\}$ es un conjunto de hechos probabilísticos y R es un conjunto de reglas. Cada hecho f_i tiene asociada una probabilidad $\theta_i \in [0, 1]$ y se asume independencia estocástica entre todos los hechos. Una *realización total* (*total choice*) es una asignación de valores de verdad a todos los hechos de F: cada f_i es verdadero o falso. Dado que hay k hechos independientes, existen 2^k realizaciones posibles. La probabilidad de una realización L que incluye los hechos f_i como verdaderos y excluye los restantes es:

$$P(L \mid DB) = \prod_{f_i \in L} \theta_i \prod_{f_i \in F \setminus L} (1 - \theta_i) \tag{1}$$

Cada realización L determina un subconjunto de hechos verdaderos que, combinado con las reglas R, produce un programa lógico definido $L \cup R$ con un modelo mínimo único $M_{DB}(L)$. De esta forma, la distribución sobre realizaciones induce una distribución sobre modelos mínimos. Sato demostró que, bajo condiciones razonables (la *condición de soporte finito*), la masa de probabilidad se concentra exclusivamente en estos modelos derivados: $P_{DB}(\text{fix}(DB)) = 1$ [2].

La probabilidad de éxito (*success probability*) de una consulta q se define como la suma de las probabilidades de todas las realizaciones cuyo modelo mínimo satisface q:

$$P(q \mid DB) = \sum_{L : M_{DB}(L) \models q} P(L \mid DB) \tag{2}$$

Para ilustrar este mecanismo de manera concreta, considérese el siguiente programa mínimo con dos hechos probabilísticos y una regla:

```
0.3::llueve.
0.5::tiene_paraguas.
se_moja :- llueve, \+tiene_paraguas.
query(se_moja).
```

Código 1: Programa ProbLog mínimo para ilustrar la semántica de distribución.

Este programa define cuatro mundos posibles ($2^2 = 4$ realizaciones de los dos hechos). La Tabla 2 detalla cada una de estas realizaciones junto con su probabilidad calculada (Ecuación 1) y su modelo mínimo resultante.

Tabla 2: Mundos posibles y modelos mínimos inducidos por la semántica de distribución. Cada fila representa una realización total de los hechos probabilísticos independientes, su probabilidad conjunta y las consecuencias lógicas derivadas.

`llueve`	`tiene_paraguas`	P(mundo)	`se_moja`	Modelo mínimo
falso	falso	$0.7 \times 0.5 = 0.35$	falso	$\emptyset$
falso	verdadero	$0.7 \times 0.5 = 0.35$	falso	{`tiene_paraguas`}
verdadero	falso	$0.3 \times 0.5 = 0.15$	verdadero	{`llueve`, `se_moja`}
verdadero	verdadero	$0.3 \times 0.5 = 0.15$	falso	{`llueve`, `tiene_paraguas`}

La probabilidad de `se_moja` es la suma de las probabilidades de los mundos donde la consulta es verdadera: $P(\texttt{se_moja}) = 0.15$. Nótese cómo la regla lógica `se_moja :- llueve, \+tiene_paraguas.` propaga la incertidumbre de los hechos hacia la consulta de manera determinista dentro de cada mundo.

3.2.2. El lenguaje ProbLog

ProbLog es un lenguaje de programación lógica probabilística que implementa la semántica de distribución [1, 11]. Su extensión sintáctica respecto a Prolog es mínima: se introduce el concepto de *hecho probabilístico*, denotado por la anotación `p::hecho.`, que indica que el hecho es verdadero con probabilidad p e independiente de todos los demás hechos probabilísticos.

Para ilustrar la sintaxis, el siguiente programa, adaptado del dominio de marketing viral utilizado por Bueno *et al.* [6] y Van den Broeck *et al.* [12], modela una red social simple donde las personas compran un producto ya sea por marketing directo o por influencia de sus contactos:

```
% Hechos probabilisticos (independientes)
0.3::compra_por_marketing(X) :- persona(X).
0.2::compra_por_influencia(X) :- persona(X).

% Hechos deterministas
persona(ana).  persona(bob).
confia(ana, bob).

% Reglas logicas
compra(X) :- compra_por_marketing(X).
compra(X) :- confia(X, Y), compra(Y),
             compra_por_influencia(X).

% Consulta
query(compra(ana)).
```

Código 2: Programa ProbLog para un dominio de marketing viral (adaptado de [6]).

En este programa, `compra_por_marketing(X)` es verdadero con probabilidad 0.3 de forma independiente para cada persona `X`. Las reglas determinan las consecuencias lógicas de estas realizaciones. ProbLog calcula $P(\texttt{compra(ana)})$ sumando sobre todos los mundos posibles en los que `compra(ana)` es derivable.

Un aspecto fundamental es que los hechos probabilísticos son estocásticamente independientes entre sí. Esto no es una limitación expresiva, ya que las dependencias entre variables se codifican mediante las reglas lógicas del programa [11].

3.2.3. Relevancia para MDP-ProbLog

La semántica de distribución es el fundamento teórico sobre el cual opera todo el mecanismo de inferencia de MDP-ProbLog. Cuando el sistema evalúa una transición $P(x'_i = 1 \mid s, a)$, lo que hace internamente es:

1. *Fijar evidencia*: Los fluentes de estado actual y la acción se inyectan como hechos con probabilidad 1.0 (verdadero) o 0.0 (falso), colapsando los mundos posibles a aquellos consistentes con el estado y acción observados.

2. *Calcular la probabilidad de éxito*: El fluente de siguiente estado $x'_i(1)$ se trata como una consulta cuya probabilidad se computa sumando sobre los mundos posibles restantes (los que aún tienen incertidumbre, proveniente de los hechos probabilísticos en las reglas de transición).

De esta manera, la sustitución de evidencia en el circuito compilado es matemáticamente equivalente a calcular la probabilidad condicional $P(x'_i \mid s, a)$ bajo la semántica de distribución, sin necesidad de aplicar explícitamente la regla de Bayes. Esta equivalencia, demostrada formalmente por Bueno *et al.* [6], es lo que permite que cada evaluación del programa ProbLog corresponda a un *backup* de Bellman (Ecuación 5).

Aunque la semántica de distribución proporciona el marco teórico para definir estas probabilidades, el cálculo directo sumando sobre mundos posibles crece exponencialmente con el número de hechos. Para resolver este problema en la práctica, ProbLog transforma la inferencia probabilística en un problema de evaluación de fórmulas booleanas ponderadas, mecanismo que se detalla a continuación.

3.3. Conteo de Modelos Ponderados

El Conteo de Modelos Ponderados es la operación computacional central que MDP-ProbLog ejecuta en cada iteración del algoritmo de Bellman para calcular las probabilidades de transición. El cálculo directo de la Ecuación 2 requiere enumerar exponencialmente muchos mundos posibles, lo cual es computacionalmente intratable en general. ProbLog aborda este problema transformando la inferencia probabilística en un problema de *Conteo de Modelos Ponderados* (WMC, por *Weighted Model Counting*) [3].

El procedimiento consta de tres pasos. Primero, se realiza el *grounding* relevante del pro-

grama respecto a la consulta, obteniendo un programa proposicional que contiene solo las cláusulas que pueden influir en el resultado. Segundo, este programa aterrizado se transforma en una fórmula booleana ponderada, donde cada variable proposicional tiene un peso asociado derivado de su probabilidad. Tercero, se calcula el conteo ponderado de los modelos de la fórmula:

$$\mathrm{WMC}(\Delta) = \sum_{\omega \models \Delta} \prod_{l \in \omega} w(l) \tag{3}$$

donde Δ es la fórmula booleana, ω recorre los modelos (asignaciones de verdad que satisfacen Δ), y $w(l)$ es el peso del literal l. Para un hecho probabilístico con probabilidad p, el peso del literal positivo es p y el del literal negativo es $1 - p$.

La probabilidad de la consulta se obtiene entonces como:

$$P(q) = \frac{\mathrm{WMC}(\Delta_q)}{\mathrm{WMC}(\Delta)} \tag{4}$$

donde Δ_q es la fórmula que incorpora la restricción de que q es verdadera. En la práctica, cuando no hay evidencia adicional, el denominador es 1 y la probabilidad se reduce al numerador.

3.4. Procesos de Decisión de Markov

Los Procesos de Decisión de Markov formalizan el problema de decisión secuencial estocástica que MDP-ProbLog está diseñado para resolver. Hasta este punto se han descrito las herramientas lógicas y probabilísticas para evaluar el estado estático de un sistema. Sin embargo, para modelar agentes que interactúan, toman decisiones y modifican su entorno a lo largo del tiempo, es necesario incorporar este paradigma matemático.

En cada paso de tiempo, un agente observa el estado actual del sistema, ejecuta una acción, recibe una recompensa y transita a un nuevo estado según una distribución de probabilidad que depende únicamente del estado actual y la acción ejecutada [5].

3.4.1. Algoritmo de Iteración de Valor

La *Iteración de Valor* (*Value Iteration*) es un algoritmo de programación dinámica que calcula V^* mediante aproximaciones sucesivas. Partiendo de una función de valor inicial arbitraria $V^{(0)}$, se construye la secuencia:

$$V^{(k+1)}(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^{(k)}(s') \right] \tag{5}$$

La secuencia $\{V^{(k)}\}$ converge a V^* bajo la norma supremo. Un criterio de convergencia comúnmente utilizado es detenerse cuando el residuo máximo satisface:

$$\max_{s \in \mathcal{S}} \left| V^{(k+1)}(s) - V^{(k)}(s) \right| \leq \frac{2\epsilon(1-\gamma)}{\gamma} \tag{6}$$

lo cual garantiza que la política *greedy* resultante es ϵ-óptima [5, 21].

Como se observa en la Ecuación 5, el cálculo requiere iterar sobre cada estado $s \in \mathcal{S}$ del sistema. En dominios complejos, representar este espacio como un conjunto plano e indivisible resulta computacionalmente inviable debido a la explosión combinatoria. Para mitigar este problema, se recurre a la factorización del espacio de estados, concepto que se detalla en la siguiente sección y que constituye el entorno estructural donde la distinción entre variables booleanas y multivaluadas adquiere su relevancia práctica.

3.5. MDPs Factorizados

La factorización del espacio de estados es la estrategia central que permite a MDP-ProbLog operar sin enumerar explícitamente todas las combinaciones posibles, y es el contexto en el que la distinción entre fluentes booleanos y multivaluados adquiere relevancia práctica. En la formulación clásica, el espacio de estados $\mathcal{S}$ se trata como un conjunto plano de $|\mathcal{S}|$ elementos. Sin embargo, en la mayoría de los dominios prácticos, el estado del sistema se describe naturalmente mediante un conjunto de variables $\mathbf{x} = (x_1, x_2, \ldots, x_n)$, donde cada variable x_i toma valores en un dominio finito $\mathrm{Dom}(x_i)$. El espacio de estados resultante es el producto cartesiano $\mathcal{S} = \mathrm{Dom}(x_1) \times \mathrm{Dom}(x_2) \times \cdots \times \mathrm{Dom}(x_n)$, cuyo tamaño crece exponencialmente con n.

En un MDP factorizado, la función de transición se descompone aprovechando las independencias condicionales entre las variables de estado. Bajo el supuesto de que el valor futuro de cada variable x_i' depende solo de un subconjunto de las variables actuales y de la acción, la probabilidad de transición se factoriza como [8]:

$$P(s' \mid s, a) = \prod_{i=1}^{n} P(x_i' \mid \mathrm{Pa}(x_i'), a) \tag{7}$$

donde $\mathrm{Pa}(x_i') \subseteq \{x_1, \ldots, x_n\}$ denota los padres de x_i' en la Red Bayesiana Dinámica (DBN) que codifica las transiciones.

Esta factorización es la que permite a algoritmos como SPUDD [9] y a MDP-ProbLog [6] operar sin enumerar explícitamente todo el espacio de estados.

3.5.1. Fluentes de estado

En el contexto de MDP-ProbLog, las variables de estado se denominan *fluentes*, un término heredado del cálculo de situaciones que denota propiedades del mundo que pueden cambiar a lo largo del tiempo. Cada fluente se estampa temporalmente mediante un argumento adicional: el fluente `vivo` en el tiempo t se representa como `vivo(t)`, y su valor en el siguiente paso temporal como `vivo(t+1)`.

En la arquitectura original de MDP-ProbLog, cada fluente es una variable de Bernoulli que toma valores en $\{0, 1\}$. Para un dominio con n fluentes booleanos, el espacio de estados tiene 2^n estados posibles. La limitación de esta representación se manifiesta cuando el dominio contiene variables con más de dos valores mutuamente excluyentes. Por ejemplo, para representar la posición de un robot en una cuadrícula de tres celdas, se requieren tres fluentes booleanos (`en_celda1`, `en_celda2`, `en_celda3`).

3.6. El Framework MDP-ProbLog

Esta sección describe la mecánica operativa del framework que será extendido en el Capítulo 4. MDP-ProbLog es un framework que combina la expresividad de ProbLog con la resolución de MDPs de horizonte infinito [6].

El solver de MDP-ProbLog implementa la Iteración de Valor (Ecuación 5) ejecutando los siguientes pasos en cada iteración: (1) para cada par estado-acción, construye un vector de evidencia que fija los valores de los fluentes actuales y la acción; (2) evalúa las probabilidades marginales de los fluentes en $t = 1$ mediante WMC sobre el circuito compilado; (3) calcula la recompensa inmediata y el valor esperado futuro; y (4) actualiza la función de valor. Este proceso se repite hasta alcanzar la convergencia.

La equivalencia formal entre ejecutar el programa probabilístico de MDP-ProbLog y realizar un *backup* de Bellman (Ecuación 5) fue demostrada por Bueno *et al.* [6], garantizando que el solver produce políticas óptimas.

3.7. Disyunciones Anotadas

Las Disyunciones Anotadas constituyen el mecanismo formal central de la extensión propuesta en esta tesina. Su capacidad de codificar la exclusión mutua a nivel semántico, sin restricciones externas, es precisamente la propiedad que permite representar fluentes de estado multivaluados de forma nativa en MDP-ProbLog. Las Disyunciones Anotadas (ADs) son un constructo sintáctico y semántico de la PLP que permite representar elecciones probabilísticas entre múltiples alternativas mutuamente excluyentes de manera nativa [7].

3.7.1. Definición formal de LPADs

Un *Logic Program with Annotated Disjunctions* (LPAD) es un conjunto de reglas de la forma:

$$(h_1 : \alpha_1) \vee (h_2 : \alpha_2) \vee \cdots \vee (h_n : \alpha_n) \leftarrow b_1, \ldots, b_m \tag{8}$$

donde los h_i son átomos, los b_j son literales que conforman el cuerpo de la regla, y los $\alpha_i \in [0, 1]$ son probabilidades tales que $\sum_{i=1}^{n} \alpha_i = 1$. Cada regla expresa que, cuando el cuerpo es verdadero, exactamente uno de los disyuntos h_i se hace verdadero, con probabilidad α_i.

En la sintaxis de ProbLog, una AD se escribe de la siguiente manera:

```
% El semaforo cambia de color con cierta probabilidad
0.4::color(verde, 1) ; 0.25::color(amarillo, 1) ;
    0.35::color(rojo, 1) :- color(verde, 0).
```

Código 3: Ejemplo de Disyunción Anotada en ProbLog.

Esta cláusula establece que si el semáforo está en verde en el tiempo $t = 0$, entonces en $t = 1$ estará en verde con probabilidad 0.4, en amarillo con probabilidad 0.25, o en rojo con probabilidad 0.35. La exclusión mutua (exactamente un color activo) está garantizada por construcción.

3.7.2. Semántica: selecciones e instancias

La semántica de un LPAD se define mediante el concepto de *selección* [7]. Una selección σ es una función que, para cada regla r del programa aterrizado, elige exactamente un par $(h_i : \alpha_i)$ de la cabeza de r. Cada selección produce una *instancia* P_σ: un programa lógico normal (sin disyunciones) donde solo el disyunto seleccionado permanece en la cabeza de cada regla.

La probabilidad de una selección se calcula asumiendo independencia entre las elecciones de distintas reglas:

$$C_\sigma = \prod_{r \in P} \sigma_{\text{prob}}(r) \tag{9}$$

donde $\sigma_{\text{prob}}(r)$ es la probabilidad del disyunto seleccionado por σ para la regla r.

La probabilidad de una interpretación I bajo el programa P es la suma de las probabilidades de todas las selecciones cuya instancia tiene a I como modelo bien fundado:

$$\pi_P^*(I) = \sum_{\sigma \in S(I)} C_\sigma \tag{10}$$

donde $S(I) = \{\sigma \mid \text{WFM}(P_\sigma) = I\}$ es el conjunto de selecciones que conducen a la interpretación I.

3.7.3. Exclusión mutua nativa

La propiedad más relevante de las ADs para este trabajo es que la exclusión mutua entre los disyuntos es una *consecuencia directa de la semántica*, no una restricción que deba imponerse externamente. Dado que cada selección elige exactamente un disyunto por regla, es imposible que dos disyuntos de la misma cláusula sean simultáneamente verdaderos en una misma instancia. Esto contrasta radicalmente con la codificación mediante hechos booleanos independientes, donde la exclusión mutua debe garantizarse mediante reglas lógicas adicionales escritas por el usuario.

Para ilustrar esta diferencia, considérese la representación de una variable con tres valores (por ejemplo, el color de un semáforo) bajo ambos enfoques:

```
% Tres fluentes booleanos independientes
state_fluent(verde).
state_fluent(amarillo).
state_fluent(rojo).

% Restricciones manuales de exclusion mutua
% (el usuario debe escribirlas explicitamente)
falso :- verde(0), amarillo(0).
falso :- verde(0), rojo(0).
falso :- amarillo(0), rojo(0).
falso :- not verde(0), not(amarillo(0)), not(rojo(0)).
```

Código 4: Codificación booleana. MDP-ProbLog original.

Como se observa, bajo este enfoque clásico el modelador está obligado a definir reglas explícitas para invalidar las combinaciones semánticamente imposibles. Específicamente, es imperativo podar del espacio de estados aquellos escenarios donde el semáforo exhiba múltiples colores de forma simultánea (e.g., verde y amarillo) o, por el contrario, aquellos donde carezca de color por completo. Esta carga representacional no solo es propensa a errores, sino que delega enteramente en el usuario la responsabilidad de mantener la coherencia lógica del modelo.

```
% Un unico fluente multivaluado con exclusion mutua nativa
state_fluent(semaforo(X)).

% La transicion se expresa como una AD:
0.5::semaforo(verde, 1); 0.45::semaforo(amarillo, 1);
    0.05::semaforo(rojo, 1) :- semaforo(verde, 0).
% No se requieren restricciones adicionales.
```

Código 5: Codificación con Disyunciones Anotadas. Extensión propuesta

En contraste, la codificación basada en Disyunciones Anotadas encapsula esta lógica de manera natural. Al declarar el fluente de estado parametrizado mediante una variable lógica

(`semaforo(X)`), se agrupan las alternativas bajo una misma entidad semántica. La regla de transición emplea una AD explícita donde la suma de las probabilidades de los disyuntos es 1. Esta sintaxis delega al motor de inferencia la garantía matemática de que, dado el estado previo, se seleccionará exactamente un color para el siguiente paso temporal.

Como resultado de esta integridad representacional, mientras que la codificación booleana genera $2^3 = 8$ combinaciones posibles de las cuales solo 3 son válidas, la codificación con ADs define un espacio exacto de 3 estados, erradicando las combinaciones espurias desde la formulación misma del modelo.

3.7.4. Interpretación causal

Vennekens, Denecker y Bruynooghe extendieron la semántica de los LPADs mediante *CP-Logic* (*Causal Probabilistic Logic*), formalizando una interpretación causal de las ADs [19]. Bajo esta interpretación, el cuerpo de cada regla representa una *causa* y los disyuntos en la cabeza son sus posibles *efectos*, cada uno con una probabilidad de ocurrencia asociada. Esta perspectiva causal mapea de manera natural al concepto de transición estocástica en un MDP: dado un estado y una acción (la causa), el siguiente valor de un fluente es uno de varios resultados mutuamente excluyentes (los efectos).

Es importante distinguir entre las probabilidades *causales* de las ADs y las probabilidades *condicionales* clásicas. Como señalan Vennekens *et al.*, si un mismo efecto puede ser producido por múltiples causas no excluyentes, la probabilidad causal de que una causa particular produzca el efecto no coincide con la probabilidad condicional del efecto dado la causa [7]. Las ADs modelan la primera, no la segunda.

3.8. Compilación de Conocimiento

La compilación de conocimiento determina la eficiencia computacional de la inferencia probabilística y es directamente afectada por la forma en que se representan las variables de estado. El cálculo del WMC (Ecuación 3) sobre una fórmula booleana arbitraria es un problema #P-completo en general [13], por lo que transformar el programa lógico en una estructura tratable es un requisito ineludible para el framework.

3.8.1. Mapa de compilación de conocimiento

Darwiche y Marquis clasificaron los principales lenguajes de compilación según sus propiedades de sucintez y las operaciones que soportan en tiempo polinomial [13]. Para la inferencia probabilística mediante WMC, las representaciones más relevantes son:

- **d-DNNF** (*Deterministic Decomposable Negation Normal Form*): Un circuito booleano en forma normal de negación donde los nodos AND tienen hijos que no comparten variables (descomponibilidad) y los nodos OR tienen hijos mutuamente excluyentes (determinismo). Sobre esta estructura, el WMC se computa en un único recorrido ascendente (*bottom-up*) del circuito.

- **SDD** (*Sentential Decision Diagram*): Introducidos por Darwiche [14], los SDDs generalizan los Diagramas de Decisión Binarios Ordenados (OBDDs) al permitir decisiones sobre fórmulas en lugar de variables individuales. Los SDDs son canónicos (dado un *vtree*, la representación es única), soportan todas las operaciones booleanas en tiempo polinomial, y son estrictamente más sucintos que los OBDDs.

ProbLog utiliza los d-DNNF como su estructura de compilación principal [15].

3.8.2. Impacto de la representación en el tamaño del circuito

Un aspecto fundamental para este trabajo es que el tamaño del circuito compilado depende no solo de la complejidad intrínseca del problema, sino también de *cómo* se formula el programa lógico. Dos programas que describen el mismo MDP pero con diferentes codificaciones pueden producir circuitos de tamaños muy distintos.

En particular, codificar una variable de k valores mediante k hechos probabilísticos booleanos independientes introduce 2^k posibles asignaciones de verdad en el espacio de búsqueda del compilador, de las cuales solo k son semánticamente válidas. Las restricciones de exclusión mutua, aunque eliminan los modelos inválidos durante la inferencia, no necesariamente reducen el tamaño de la estructura compilada, ya que el compilador puede necesitar representar explícitamente la estructura lógica que descarta esas combinaciones.

En contraste, una Disyunción Anotada con k disyuntos codifica directamente las k alternativas válidas con la restricción $\sum \alpha_i = 1$ integrada en la semántica. ProbLog implementa las ADs internamente mediante nodos `choice` que garantizan estructuralmente la exclusión mutua, lo cual tiene el potencial de producir circuitos más compactos al evitar la representación de combinaciones imposibles desde el origen.

El impacto real de esta transición representacional sobre el tamaño de los circuitos generados y los tiempos de inferencia constituye una de las preguntas centrales de esta investigación. Este comportamiento se analizará y cuantificará empíricamente en el Capítulo de Experimentación y Resultados.

En síntesis, este capítulo ha establecido la cadena completa de conceptos formales que sustenta la extensión propuesta. La programación lógica proporciona el lenguaje de representación, la semántica de distribución y el WMC proporcionan el mecanismo de inferencia, los MDPs factorizados definen el problema que se resuelve, y las Disyunciones Anotadas ofrecen la herramienta formal para codificar la exclusión mutua de manera nativa. MDP-ProbLog integra todos estos elementos en un framework operativo cuya única limitación estructural pendiente, la restricción a fluentes booleanos, es precisamente el objeto de la extensión que se detalla en el siguiente capítulo.

4. Diseño e Implementación

Una vez establecidos los fundamentos teóricos de la Programación Lógica Probabilística, los Procesos de Decisión de Markov factorizados y las Disyunciones Anotadas, el presente capítulo aborda el diseño y la construcción de la extensión al núcleo de MDP-ProbLog. Se analiza primero la arquitectura original del sistema para identificar sus limitaciones estructurales en la representación del espacio de estados. A partir de esta evaluación, se detalla el diseño de las nuevas estructuras de datos y los mecanismos de inferencia de tipos, seguido por las modificaciones al motor de inferencia y al algoritmo de resolución. Finalmente, se demuestra cómo la extensión preserva la retrocompatibilidad con los modelos booleanos preexistentes.

4.1. Análisis de la arquitectura original

Para determinar los puntos de intervención de la extensión propuesta, es necesario examinar cómo el framework MDP-ProbLog procesaba la información en su diseño original [6]. El sistema opera bajo una arquitectura modular que actúa como capa intermedia entre el motor de inferencia probabilística de ProbLog y un algoritmo de resolución numérica.

4.1.1. Componentes del sistema original

La arquitectura original se compone de cuatro módulos con responsabilidades delimitadas:

`engine.py` Implementa un adaptador sobre el motor interno de ProbLog (`DefaultEngine`). Encapsula la Base de Datos de Cláusulas (*ClauseDB*) y expone métodos para añadir hechos lógicos, ejecutar el grounding relevante y compilar circuitos evaluables para el Conteo de Modelos Ponderados.

`fluent.py` Define las abstracciones del estado del mundo. Las clases `Fluent` y `StateSpace` generan e indexan las configuraciones posibles del sistema, asumiendo un espacio factorizado puramente booleano.

`mdp.py` Actúa como entidad coordinadora del framework. Interroga la base de datos de cláusulas buscando predicados reservados (`state_fluent/1`, `action/1`, `utility/2`), coordina la incorporacion de hechos y traduce las relaciones lógicas en una semántica de transiciones de Markov.

`value_iteration.py` Implementa el algoritmo de Iteración de Valor. Opera sobre índices enteros y distribuciones marginales para aproximar la ecuación de optimalidad de Bellman (Ecuación 5).

4.1.2. Flujo de ejecución

La interacción entre estos componentes sigue un flujo secuencial de cinco fases:

1. **Análisis y carga.** El módulo `engine.py` realiza el *parsing* del archivo Prolog proporcionado por el usuario y lo inyecta en la `ClauseDB`.

2. **Descubrimiento del dominio.** El coordinador (`mdp.py`) interroga la base de datos para identificar los fluentes de estado y las acciones, e instancia los iteradores correspondientes en `fluent.py`.

3. **Inyección temporal.** Para modelar la dinámica secuencial, `mdp.py` transforma las declaraciones abstractas agregando estampas de tiempo ($t = 0$ para el estado actual, $t = 1$ para el siguiente estado). Simultáneamente, introduce hechos probabilísticos neutrales en $t = 0$ para permitir la sustitución posterior de evidencia durante la evaluación.

4. **Aterrizado y compilación.** Se ejecuta un *grounding* relevante respecto a las consultas de interés. Las reglas de transición se compilan en un circuito proposicional tratable (SDD o d-DNNF).

5. **Resolución.** El módulo `value_iteration.py` recorre el espacio de estados, inyecta cada par estado-acción como evidencia determinista en el motor, y solicita el cálculo de las probabilidades marginales $P(s' \mid s, a)$ mediante WMC para actualizar la función de valor $V(s)$.

Esta separación de responsabilidades facilitó la tarea de realizar modificaciones y constuir sobre el un solver funcional y matemáticamente correcto.

4.1.3. Limitaciones en la gestión de fluentes

Aunque MDP-ProbLog permite modelar variables de estado relacionales mediante el predicado `state_fluent/1`, su diseño interno asume que cada instancia aterrizada (*ground atom*) opera como una variable de Bernoulli estocásticamente independiente.

El módulo `fluent.py` original implementaba la clase `StateSpace` como un iterador combinatorio plano. Dado un conjunto de n fluentes booleanos, el sistema generaba el producto cartesiano completo, produciendo un espacio de estados de tamaño 2^n. Esta arquitectura resulta funcional para dominios puramente binarios, pero genera un desajuste al modelar fenómenos con estados representados con variables categoricas o multivaluadas.

Esta divergencia entre la semántica del dominio y la representación computacional genera dos consecuencias negativas. Por un lado, delega en el usuario la responsabilidad de escribir reglas lógicas adicionales para bloquear transiciones hacia los estados inválidos, lo que aumenta la complejidad del modelado y la propensión a errores. Por otro lado, el algoritmo de Iteración de Valor calcula respaldos de Bellman para estados inalcanzables, consumiendo recursos computacionales sin contribuir a la solución.

4.1.4. Identificación de puntos de extensión

El análisis de las limitaciones expuestas revela que la solución no requiere alterar el motor de inferencia probabilística subyacente de ProbLog. La compilación a circuitos lógicos y el WMC permanecen como la estrategia de evaluación del framework. La intervención se localiza en la capa intermedia donde las declaraciones lógicas se transforman en variables de estado evaluables para el solver. Se identificaron tres puntos de extensión:

1. **Representación del espacio de estados.** Reemplazar la representación por una estructura factorizada que admita factores de cardinalidad arbitraria.

2. **Inyección de hechos en la ClauseDB.** Adaptar el mecanismo de inyección temporal para que los fluentes multivaluados se representen como Disyunciones Anotadas en lugar de hechos probabilísticos independientes.

3. **Cálculo del valor esperado.** Reestructurar la evaluación del valor futuro en la Iteración de Valor para que recorra los factores del esquema en lugar de enumerar combinaciones binarias.

Al acotar las modificaciones a estos tres puntos, se preservan las garantías de inferencia exacta de ProbLog. La siguiente sección detalla el diseño formal de cada componente de la extensión.

4.2. Diseño de la extensión

Esta sección detalla el diseño conceptual de la extensión propuesta. El objetivo es transitar de un modelo de variables booleanas independientes hacia una estructura factorizada que soporte fluentes de estado multivaluados de forma nativa. Para lograrlo, se diseñaron dos componentes centrales: una estructura de datos factorizada (`FluentSchema`) y un mecanismo de clasificación automática (`FluentClassifier`).

4.2.1. FluentSchema: estructura factorizada del espacio de estados

El primer componente del rediseño reemplaza el almacenamiento de fluentes basado en listas planas por una estructura de datos centralizada denominada `FluentSchema`. Esta clase define el espacio de estados como el producto cartesiano de factores lógicamente independientes, donde cada factor pertenece a una de dos categorías:

- **Booleano (base 2).** Una variable binaria independiente que toma valores en $\{0, 1\}$. Corresponde a los fluentes tradicionales de MDP-ProbLog.

- **Multivaluado (base N).** Un grupo de N opciones mutuamente excluyentes donde exactamente una está activa en cada estado. La codificación interna utiliza representación *one-hot* sobre las opciones del grupo.

El esquema mantiene internamente tres estructuras de datos: la lista ordenada de factores (donde cada factor es una lista de términos atemporales), la lista de bases correspondientes, y

una lista aplanada de todos los términos en orden de registro. Los strides para la indexación mixed-radix se calculan bajo demanda y se almacenan en caché.

Para gestionar los factores multivaluados, el esquema implementa un mecanismo de agrupamiento basado en el nombre del predicado (*functor*). Al instanciar el programa, el sistema agrupa los términos multivaluados evaluando su functor. Esto garantiza, por ejemplo, que los valores `pos(a)`, `pos(b)` y `pos(c)` se agrupen como alternativas excluyentes del mismo factor. Esta convención constituye una de las limitaciones documentadas en la sección de Alcances (1.5.2): todos los valores de un fluente multivaluado deben compartir el mismo nombre de predicado.

4.2.2. FluentClassifier: mecanismo de clasificación automática

Para que el esquema factorizado se construya correctamente sin obligar al usuario a declarar manualmente el tipo de cada fluente en todos los casos, se diseñó el módulo `FluentClassifier`. Este componente ejecuta un proceso de clasificación en tres etapas: validación estática de las declaraciones, registro e inferencia de tipos, y ensamblaje del esquema.

El clasificador admite dos modos de declaración complementarios:

- **Modo implícito (`state_fluent/1`):** El *framework* infiere automáticamente el tipo del fluente analizando el origen de sus valores aterrizados (*grounded values*).
- **Modo explícito (`state_fluent/2`):** El usuario especifica directamente una etiqueta de tipo (`bool` o `multivalued`), otorgando control total sobre la clasificación e ignorando la inferencia automática.

Algoritmo 1 Inferencia de tipo por intersección de orígenes estocásticos

Entrada: Conjunto de términos aterrizados G de aridad a; índice invertido $\mathcal{I} : \Sigma \to \mathcal{P}(\Gamma)$ que mapea valores Σ a conjuntos de grupos AD Γ.
Salida: Clasificación de tipo: `bool` o `multivalued`.

```
if a = 0 then return bool          ▷ Los fluentes proposicionales son siempre booleanos.
end if
for p ← 1 a a do
    V_p ← {arg_p(g) | g ∈ G}                ▷ Extraer conjunto de valores en la posición p.
    C ← ⋂_{v∈V_p} I(v)                      ▷ Intersección de grupos AD para todos los valores.
    if C ≠ ∅ then return multivalued ▷ Todos los valores provienen de la misma AD.
    end if
end for
    return bool                             ▷ Ninguna posición proviene íntegramente de una AD.
```

Durante el análisis estático, el clasificador aplica reglas de validación estrictas para prevenir errores de modelado. Por ejemplo, si un grupo multivaluado resulta con una cardinalidad inferior a dos opciones, se interrumpe la ejecución con un `FluentCardinalityError`, ya que

un grupo mutuamente excluyente requiere al menos dos estados válidos para contribuir al espacio mixed-radix. Si un fluente se declara en ambos modos, el sistema emite una advertencia de duplicidad y prioriza la declaración explícita.

Reglas de inferencia implícita. En el modo implícito, la decisión de clasificación depende directamente de la aridad del término lógico y de la estructura de la Base de Datos de Cláusulas:

1. **Aridad 0:** Los fluentes proposicionales carecen de argumentos para generar un dominio de valores; por lo tanto, siempre se clasifican como booleanos .

2. **Aridad ≥ 1:** El clasificador verifica si los valores aterrizados en alguna posición argumental provienen íntegramente de una Disyunción Anotada (AD). Si la intersección de estos valores con el vocabulario de alguna AD declarada es exhaustiva, el fluente se clasifica como `multivalued`; en caso contrario, si provienen de hechos estáticos ordinarios, se clasifica como `bool`.

El Algoritmo 1 formaliza esta lógica de decisión por intersección.

Agrupamiento y el problema del Colapso Estructural. Una vez que un conjunto de términos implícitos es identificado como multivaluado, el sistema consolida el factor basándose exclusivamente en el nombre del predicado (*functor*) . Bajo esta convención, el framework interpreta los fluentes como una única variable categórica global por cada nombre de predicado.

Si bien este comportamiento es eficiente para dominios simples, introduce un riesgo documentado como *Colapso Estructural* . Este fenómeno ocurre cuando el modelador intenta representar familias de variables indexadas por entidad (por ejemplo, `pos(robot1, a)` y `pos(robot2, b)`) utilizando el modo implícito. Al compartir el mismo *functor* (`pos`), el clasificador las agrupa erróneamente dentro de un único factor mutuamente excluyente, destruyendo la independencia estocástica entre los distintos robots e invalidando la semántica del dominio .

Justificación del modo explícito. Es precisamente el riesgo de colapso estructural lo que fundamenta la necesidad arquitectónica del predicado `state_fluent/2` con la etiqueta `multivalued`. El modo explícito desactiva el agrupamiento ciego por functor, permitiendo al modelador definir reglas parametrizadas que instancien múltiples factores multivaluados verdaderamente independientes. Esta dualidad de diseño garantiza que el framework mantenga una sintaxis ligera para variables globales, sin sacrificar la expresividad relacional necesaria para dominios multi-agente complejos.

4.2.3. Codificación *mixed-radix* del espacio de estados

Con los fluentes correctamente clasificados y agrupados en el `FluentSchema`, el sistema debe iterar y enumerar el nuevo espacio de estados sin generar combinaciones inválidas. En la

arquitectura original, la enumeración utilizaba un contador binario que asignaba un índice entero a cada combinación asumiendo base 2 para todos los factores. Al introducir variables con cardinalidad mayor a 2, se requiere un sistema de numeración capaz de manejar bases heterogéneas.

La clase `FactorSpace` (de la cual heredan `StateSpace` y `ActionSpace`) implementa una codificación de base mixta (*mixed-radix*). Bajo este esquema, el espacio de estados se define como el producto cartesiano de m factores independientes, donde cada factor k posee una base b_k correspondiente a su cardinalidad.

El tamaño total del espacio de estados se define como el producto de todas las bases:

$$|\mathcal{S}| = \prod_{k=1}^{m} b_k \tag{11}$$

Para mapear cada estado a un índice entero $I \in [0, |\mathcal{S}| - 1]$, se calcula un stride S_k para cada factor, definido como el producto de las bases de los factores precedentes:

$$S_k = \prod_{i=1}^{k-1} b_i \quad \text{con } S_1 = 1 \tag{12}$$

Dada una configuración donde cada factor k toma un valor local $v_k \in [0, b_k - 1]$, el índice global se calcula como:

$$I = \sum_{k=1}^{m} (v_k \times S_k) \tag{13}$$

La operación inversa (decodificación de un índice a una valuación) se realiza extrayendo sucesivamente el residuo módulo la base de cada factor:

$$v_k = I \text{ mód } b_k, \quad I \leftarrow \left\lfloor \frac{I}{b_k} \right\rfloor \tag{14}$$

Ambas operaciones eliminan la generación de estados imposibles, ya que el espacio iterado corresponde exactamente a las combinaciones semánticamente válidas del modelo.

4.2.4. Sintaxis de usuario para fluentes multivaluados

El propósito último de reestructurar la representación interna del *framework* es proporcionar al modelador una interfaz declarativa más natural, robusta y matemáticamente segura. La extensión permite declarar variables categóricas y especificar sus transiciones estocásticas directamente mediante la sintaxis nativa de las Disyunciones Anotadas (AD) de ProbLog, eliminando la necesidad de gestionar la exclusión mutua de forma manual.

Para ilustrar el impacto de este rediseño a nivel de código, considérese un dominio [Pensar bien que dominio presentar, algo sencillo]

Código 6: Modelado de posición en la arquitectura original.

En el Código 6, la semántica del dominio diverge severamente de su representación computacional. Al tratar cada posición como una variable de Bernoulli independiente, el motor de inferencia asume un espacio de estados de tamaño $2^4 = 16$ combinaciones, de las cuales solo 4 son semánticamente válidas . El modelador asume la carga de escribir reglas lógicas adyacentes para asegurar que la probabilidad fluya correctamente y evitar transiciones hacia configuraciones imposibles (por ejemplo, el agente existiendo simultáneamente en `pos(a)` y `pos(b)`).

Con la extensión desarrollada, el mismo modelo se especifica explotando la semántica de las Disyunciones Anotadas.

```
% 1. Declaracion unificada (Genera 4 estados bajo codificacion
   one-hot)
state_fluent(pos(X), multivalued) :- loc(X).
loc(a). loc(b). loc(c). loc(d).

% 2. Transicion estocastica nativa (Garantiza suma de
   probabilidades = 1.0)
0.8::pos(b, 1); 0.2::pos(a, 1) :- pos(a, 0), move_right.

% 3. (No se requieren reglas de exclusion mutua)
```

Código 7: Modelado de posición utilizando fluentes multivaluados.

Como se observa en el Código 7, la nueva arquitectura permite una declaración explícita parametrizada que colapsa las cuatro posiciones en un único factor de cardinalidad 4. Las reglas de transición codifican directamente la dinámica de Markov utilizando las cabezas múltiples de la Disyunción Anotada.

Este enfoque presenta tres ventajas arquitectónicas fundamentales:

1. **Precisión dimensional:** El espacio de estados se reduce de 2^N a N combinaciones por factor, iterando exclusivamente sobre configuraciones válidas mediante la codificación *mixed-radix*.

2. **Seguridad probabilística:** La restricción estocástica $\sum \alpha_i = 1$ es garantizada por construcción por el analizador de ProbLog para las ADs.

3. **Mantenibilidad:** Se elimina la necesidad de redactar y depurar reglas manuales de exclusión mutua, reduciendo drásticamente la verbosidad del programa lógico.

4.3. Implementación

Esta sección describe las modificaciones realizadas sobre los módulos del framework para materializar el diseño expuesto en la sección anterior. Las adaptaciones se concentran en tres componentes: el adaptador al motor de inferencia, el coordinador central del MDP, y el algoritmo de resolución.

4.3.1. Adaptador al motor de inferencia

La clase `Engine` actúa como la interfaz entre MDP-ProbLog y el núcleo de ProbLog. Para soportar fluentes multivaluados, se incorporaron dos métodos nuevos.

Inyección de Disyunciones Anotadas. El método `add_annotated_disjunction` permite añadir dinámicamente una AD en la `ClauseDB`. Recibe un grupo de hechos mutuamente excluyentes junto con sus probabilidades, construye internamente un objeto `AnnotatedDisjunction` de ProbLog, y lo incorpora a la base de datos. El método retorna los identificadores de los nodos `choice` resultantes, que representan las ramas estocásticas de la disyunción dentro del circuito.

Construcción del índice invertido. El método `get_ads_inverted_index` recorre linealmente la tabla de instrucciones de la `ClauseDB`, identificando todos los nodos de tipo `choice`. Para cada nodo detectado, extrae el término lógico contenido en la estructura interna del nodo y registra sus argumentos en un diccionario que mapea cada valor concreto (como cadena de texto) al conjunto de identificadores de grupo de la AD que lo generó. El resultado es un índice invertido con la estructura:

$$\{\texttt{valor_string} \rightarrow \{\texttt{group_id_1, group_id_2, ...}\}\}$$

Este índice es consumido por el `FluentClassifier` durante la fase de inferencia de tipos, como se describió en 4.2.2. La construcción del índice se apoya en atributos internos de los nodos `choice` de ProbLog (específicamente `node.functor.args[2]` y `node.group`), lo cual constituye una dependencia sobre la representación interna de ProbLog documentada como limitación conocida.

4.3.2. Coordinador central del MDP

La clase `MDP` coordina el flujo completo que transforma el programa lógico del usuario en un problema de optimización resoluble. Para integrar el esquema factorizado, el método de inicialización `__prepare()` se reestructuró en cinco etapas secuenciales:

1. **Clasificación de fluentes.** Se instancia el `FluentClassifier`, que consume las declaraciones del programa y el índice invertido de ADs para construir el `FluentSchema`.

2. **Inyección del estado actual ($t = 0$).** Para cada factor del esquema, se inyectan hechos que representan el estado actual. Los factores booleanos se inyectan como hechos probabilísticos con probabilidad neutral de 0.5 mediante `add_fact`. Los factores

multivaluados se inyectan como Disyunciones Anotadas con probabilidades uniformes $1/N$ mediante `add_annotated_disjunction`. En ambos casos, las probabilidades son neutrales y serán sustituidas por los valores de evidencia (0 o 1) durante la evaluación.

3. **Inyección de acciones.** Las acciones declaradas en el programa se inyectan como una única Disyunción Anotada con distribución uniforme, siguiendo el mismo principio de exclusión mutua nativa.

4. **Grounding relevante.** Se construye la lista de consultas combinando las utilidades, los fluentes de siguiente estado ($t = 1$), las acciones y los fluentes del estado actual ($t = 0$). La inclusión de los fluentes $t = 0$ en esta lista es una decisión técnica que garantiza que ProbLog preserve sus identificadores durante el proceso de optimización del grounding, permitiendo localizarlos posteriormente al sustituir evidencia.

5. **Compilación.** Los fluentes de siguiente estado y las utilidades se compilan en nodos del circuito evaluable, generando los mapas término→nodo que se reutilizan en cada evaluación posterior.

4.3.3. Adaptaciones al algoritmo de resolución

La adopción de una representación mixed-radix exige que el cálculo del valor esperado futuro durante las actualizaciones de Bellman sea capaz de navegar la estructura factorizada. En la implementación original, este cálculo enumeraba las combinaciones binarias posibles. La nueva implementación ejecuta un recorrido recursivo sobre los factores del esquema.

Algoritmo 2 Cálculo Expected Value

Entrada: Profundidad k, índice global acumulado I, probabilidad conjunta P_{joint}.
Entrada: Secuencia de factores estructurados $\mathcal{F}$, vector de strides S, función de valor V.
Entrada: Umbral de filtrado probabilístico ϵ.
Salida: Valor esperado escalar E para el subárbol.

```
function ExpectedValue(k, I, P_joint)
    if k = |F| then                    ▷ Caso base: Se han procesado todos los factores. return P_joint × V[I]
    end if
    E ← 0
    for cada tupla (v, p) ∈ F_k do                ▷ Iterar ramas activas del factor actual.
        if p > ε then                             ▷ Filtrado de ramas irrelevantes .
            i_local ← IndexLocal(F_k, v)          ▷ Decodificación del índice local.
            I_next ← I + (i_local × S_k)          ▷ Acumulación posicional mixed-radix.
            P_next ← P_joint × p                  ▷ Acumulación de probabilidad conjunta.
            E ← E + ExpectedValue(k + 1, I_next, P_next)   ▷ Llamada recursiva.
        end if
    end for
    return E
end function
```

En cada nivel de recursión k, correspondiente a un factor del `FluentSchema`, el algoritmo extrae las ramas activas del factor (los posibles valores futuros con sus probabilidades marginales). Para cada rama, calcula el índice global del estado sucesor acumulando el producto del índice local por el stride posicional del factor k, y multiplica la probabilidad conjunta del camino. Al alcanzar el nivel base (todos los factores procesados), el índice acumulado identifica unívocamente el estado destino y se retorna el producto de la probabilidad conjunta por el valor almacenado $V[\text{índice}]$.

Adicionalmente, el método `structured_transition` del MDP aplica un filtro previo sobre cada factor, descartando las ramas cuya probabilidad sea inferior a un umbral configurable (10^{-6} por defecto). Esto reduce el número de ramas activas por factor y, en consecuencia, la profundidad efectiva del árbol recursivo. En transiciones deterministas o cuasi-deterministas, donde un solo valor concentra la totalidad de la masa probabilística, el recorrido se reduce a una sola rama por factor.

4.3.4. Herramientas de verificación y exportación

Para facilitar la verificación de los modelos resueltos, se desarrollaron dos módulos complementarios.

El `CSVExporter` permite exportar las estructuras del MDP y los resultados de la Iteración de Valor a archivos CSV. Las exportaciones disponibles incluyen la matriz de transición $P(s' \mid s, a)$ expandida a partir de las transiciones factorizadas, la función de recompensa $R(s, a)$, la función de valor óptima $V^*(s)$, la política óptima $\pi^*(s)$, la tabla de valores Q (si fue calculada), y el historial de convergencia (si fue registrado). La expansión de las transiciones factorizadas a formato plano se realiza mediante una recursión análoga a la del cálculo del valor esperado, generando el producto cartesiano de las ramas activas por factor.

El `Simulator` permite ejecutar trayectorias de simulación a partir de un estado inicial, siguiendo la política calculada. En cada paso de tiempo, el simulador selecciona la acción prescrita por la política, obtiene la distribución de transición factorizada del motor, y muestrea el siguiente estado mediante selección categórica ponderada sobre cada factor del esquema.

Ambas herramientas operan sobre la misma representación factorizada que utiliza el solver, garantizando consistencia entre la resolución y la inspección del modelo.

4.4. Retrocompatibilidad

La extensión fue diseñada para preservar el funcionamiento de los modelos booleanos existentes sin requerir modificaciones por parte del usuario. Esta garantía se sustenta en el comportamiento del `FluentClassifier` ante programas que no contienen Disyunciones Anotadas.

Cuando el clasificador procesa un programa Prolog que define fluentes mediante `state_fluent/1` sin que existan ADs en el programa, el índice invertido no contiene entradas relevantes y la inferencia de tipos no detecta ninguna posición estocástica. En consecuencia, todos los fluentes se clasifican como booleanos (base 2). Bajo esta clasificación, el cálculo de strides produce la

serie de potencias binarias $\{1, 2, 4, 8, \ldots\}$, la incorporacion temporal recurre a hechos probabilísticos individuales con probabilidad neutral 0.5, y el espacio de estados se enumera como el producto cartesiano binario. El comportamiento resultante es idéntico al de la arquitectura original.

Esta propiedad permite al usuario adoptar la sintaxis multivaluada de manera incremental, incorporando fluentes multivaluados en modelos existentes sin necesidad de reescribir las partes booleanas del programa.

Con la arquitectura, los algoritmos y la implementación definidos, el sistema cuenta con la capacidad de modelar y resolver MDPs con fluentes multivaluados de manera nativa. El siguiente capítulo somete esta extensión a una evaluación empírica mediante casos de estudio que cuantifican el impacto de la reestructuración tanto en la simplificación del modelado como en el comportamiento computacional del solver.

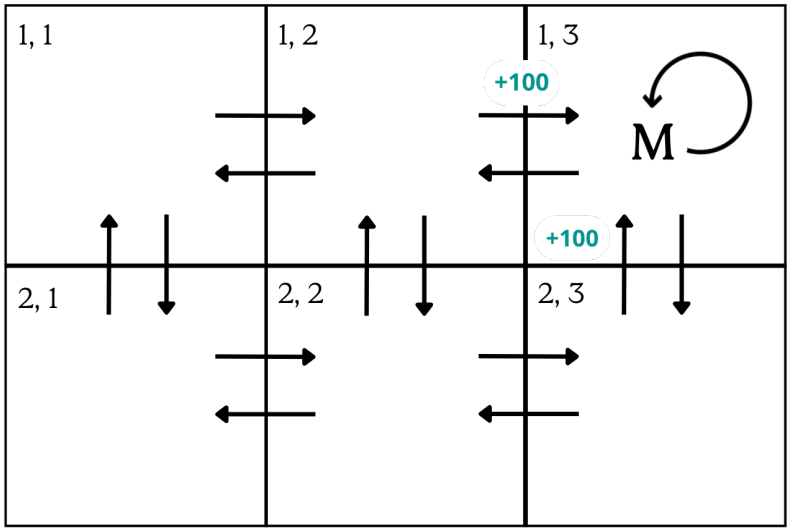
5. Experimentación y Resultados

El capítulo anterior presentó el diseño y la implementación de la extensión al framework MDP-ProbLog. El presente capítulo somete dicha extensión a una evaluación empírica mediante un conjunto de experimentos diseñados para responder tres preguntas. Primero, si la extensión produce soluciones correctas. Segundo, si simplifica el proceso de modelado. Tercero, cómo afecta al comportamiento computacional del solver. Para ello se utiliza el grid de Mitchell como dominio de referencia, variando el tamaño del grid para analizar las tendencias de escalamiento bajo dos codificaciones del espacio de estados.

5.1. Diseño experimental

5.1.1. Dominio de prueba

El dominio de prueba es una familia de grids basada en el grid world de Mitchell [22], un problema canónico en la literatura de MDPs utilizado para ilustrar el algoritmo de Iteración de Valor. En este dominio, un agente se desplaza por una cuadrícula rectangular de r filas y c columnas. En cada paso de tiempo, el agente selecciona una de cuatro acciones (arriba, abajo, izquierda, derecha). Las transiciones son deterministas: la acción produce el movimiento esperado siempre que la celda destino exista dentro de los límites del grid. Si el agente intenta moverse contra una pared, permanece en su celda actual. El grid contiene una celda meta que otorga una recompensa positiva al ser alcanzada.



Acciones: izquierda, derecha, arriba, abajo y quedarse.

Figura 1: Entorno de navegación en cuadrícula utilizado para la validación de la política óptima. Adaptado de Mitchell [22].

Se seleccionaron cinco tamaños de grid para el análisis de escalamiento. La Tabla 3 resume sus características. La selección sigue tres criterios: incluir el grid original de Mitchell (2×3)

como punto de verificación contra la solución publicada, cubrir un rango de tamaños que permita observar tendencias de escalamiento, e incluir el grid 4×4 como punto de control donde la codificación binaria no produce estados espurios ($16 = 2^4$).

Tabla 3: Tamaños de grid seleccionados para la evaluación experimental. La columna "Bits" indica el número de fluentes booleanos requeridos por la codificación binaria. Los estados espurios son combinaciones binarias que no corresponden a ninguna celda del grid.

#	Grid	Celdas	Bits	$\lvert\mathcal{S}\rvert_{\text{bin}}$	$\lvert\mathcal{S}\rvert_{\text{fact}}$	Espurios
1	2×3	6	3	8	6	2
2	3×3	9	4	16	9	7
3	4×4	16	5	32	16	16
4	5×5	25	5	32	25	7
5	6×6	36	6	64	36	28

5.1.2. Codificaciones comparadas

Para cada tamaño de grid se construyeron dos formulaciones del mismo MDP, que difieren exclusivamente en la representación del espacio de estados.

Codificación binaria. La posición del agente se codifica mediante $\lceil \log_2(r \times c) \rceil$ fluentes booleanos. Cada combinación de valores binarios representa un estado del grid, y las reglas de transición se escriben explícitamente para cada par (estado, acción), activando los bits correspondientes al estado destino. Cuando el número de celdas no es una potencia de 2, la codificación produce estados espurios que el modelo debe contemplar mediante reglas adicionales.

```
% 1. Declaracion de fluentes (log2(6) -> 3 bits, genera 8
   combinaciones)
state_fluent(b1).
state_fluent(b2).
state_fluent(b3).

% 2. Manejo de estados espurios (000 y 111 son invalidos en el
   grid 2x3)
blocked :- b1(0), b2(0), b3(0).
blocked :- not(b1(0)), not(b2(0)), not(b3(0)).


% 3. Transiciones representativas
% Movimiento hacia la derecha desde (1,1) [codificado como 1,0,0]
% hacia (1,2) [codificado como 0,1,0]. Se activa b2.
1.0::b2(1) :- b1(0), not(b2(0)), not(b3(0)), right, not(blocked).

% Movimiento hacia abajo desde (1,2) [codificado como 0,1,0]
```

```
% hacia (2,2) [codificado como 1,0,1]. Se activan b1 y b3.
1.0::b1(1) :- not(b1(0)), b2(0), not(b3(0)), down.
1.0::b3(1) :- not(b1(0)), b2(0), not(b3(0)), down.
```

Código 8: Fragmento del modelo binario de control para el grid de 2×3.

Como se observa en el Código 8, la representación binaria desvincula la lógica espacial del problema de su implementación computacional. El motor de inferencia asume la existencia de $2^3 = 8$ estados, obligando al modelador a declarar el predicado `blocked` para excluir las combinaciones `000` y `111` de la evaluación de recompensas y transiciones. Además, transitar hacia un estado que requiere múltiples bits activos exige redactar múltiples cabezas de regla independientes para la misma condición de origen.

Codificación multivaluada factorizada. La posición se codifica mediante dos fluentes multivaluados: `row(i)` con base r y `col(j)` con base c. El espacio de estados resultante tiene exactamente $r \times c$ estados. Las transiciones se expresan en términos de coordenadas. No se requieren reglas de exclusión mutua.

```
% 1. Conocimiento de fondo (Background knowledge)
row(1). row(2).
col(1). col(2). col(3).

% 2. Declaracion de fluentes (Genera exactamente 2x3 = 6 estados
   validos)
state_fluent(x(X), multivalued) :- row(X).
state_fluent(y(Y), multivalued) :- col(Y).

% 3. Transiciones parametrizadas por factor
% Factor Y (columna): El movimiento a la derecha incrementa Y
1.0::y(Y_new, 1) :- y(Y, 0), right, Y_new is Y + 1, col(Y_new),
   not(terminal).
% Choque con la pared: Y se mantiene igual
1.0::y(Y, 1)     :- y(Y, 0), right, Y_new is Y + 1, not(col(Y_new
   )), not(terminal).

% Factor X (fila): El movimiento hacia arriba decrementa X
1.0::x(X_new, 1) :- x(X, 0), up, X_new is X - 1, row(X_new), not(
   terminal).
```

Código 9: Fragmento del modelo multivaluado factorizado para el grid de 2×3.

Como se evidencia en el Código 9, la representación multivaluada factorizada alinea perfectamente la semántica espacial del problema con su implementación lógica. Al definir los fluentes `x(X)` y `y(Y)` explícitamente como `multivalued`, el *framework* construye un espacio *mixed-radix* de base 2×3, iterando exclusivamente sobre los 6 estados semánticamente posibles. Esto elimina por completo la necesidad de codificar reglas de bloqueo (`blocked`) para

estados espurios.

Además, el uso de variables lógicas y aritmética (`Y_new is Y + 1`) permite generalizar la dinámica de transición. En lugar de enumerar reglas para cada celda individual como en la codificación binaria, un único par de reglas rige el comportamiento de toda una acción a lo largo del tablero. Esta generalización reduce significativamente la verbosidad del modelo, facilitando su escalabilidad a *grids* de dimensiones arbitrarias sin requerir modificaciones en la lógica subyacente.

5.1.3. Métricas de evaluación

Las métricas se organizan en tres grupos correspondientes a las tres preguntas del experimento.

Las métricas de *corrección funcional* verifican que ambas codificaciones produzcan la misma función de valor óptima $V^*(s)$ y la misma política $\pi^*(s)$, y que estos resultados coincidan con la solución de referencia. La comparación numérica utiliza una tolerancia de 10^{-6}.

Las métricas de *complejidad del modelo* cuantifican el esfuerzo de modelado: número de fluentes declarados, número de cláusulas del programa, número de reglas de exclusión mutua o manejo de estados espurios, y tamaño del espacio de estados $|\mathcal{S}|$.

Las métricas de *impacto computacional* miden el tiempo de preparación del MDP (clasificación, inyección, grounding, compilación), el tiempo de Iteración de Valor, y el tiempo total de resolución.

5.1.4. Protocolo de medición

Las métricas de corrección y complejidad son deterministas y se midieron una sola vez por configuración. Las mediciones de tiempo se ejecutaron en $N = 10$ repeticiones independientes por configuración. Cada repetición consistió en una ejecución completa del pipeline.

5.1.5. Entorno de pruebas

Los experimentos se ejecutaron sobre un equipo con procesador Intel Core Ultra 7 155U (14 núcleos), 16 GiB de memoria RAM y sistema operativo Ubuntu 24.04.4 LTS (kernel Linux 6.17.0, arquitectura x86_64). El intérprete utilizado fue Python 3.12.3 con ProbLog 2.2.10. Se emplearon los backends de compilación d-DNNF y SDD disponibles en ProbLog. Los parámetros del solver se fijaron en $\gamma = 0.9$ (factor de descuento), $\epsilon = 0.01$ (tolerancia de convergencia) y $\epsilon_{\text{thr}} = 10^{-6}$ (umbral de filtrado de ramas con probabilidad despreciable). Estos parámetros se mantuvieron constantes en todas las configuraciones experimentales.

5.2. Verificación de corrección funcional

El primer objetivo de la evaluación es demostrar que la extensión multivaluada produce resultados correctos. Para ello se comparan las soluciones obtenidas por ambas codificaciones entre sí y contra la solución de referencia.

La Tabla 4 presenta los resultados de la verificación cruzada entre ambas codificaciones. Para cada tamaño de grid, se comparan los valores numéricos de $V^*(s)$ obtenidos por la codificación binaria y la codificación factorizada.

Tabla 4: Verificación de corrección funcional. La columna V^* indica si las funciones de valor de ambas codificaciones coinciden numéricamente (tolerancia 10^{-6}). La diferencia máxima observada fue 0.0 en todos los casos.

Grid	V^* **coincide**	**Dif. máxima**	**Iter. binaria**	**Iter. factorizada**
2×3	Sí	0.0	3	3
3×3	Sí	0.0	5	3
4×4	Sí	0.0	7	4
5×5	Sí	0.0	9	5
6×6	Sí	0.0	11	6

Ambas codificaciones producen funciones de valor idénticas para los cinco tamaños evaluados, con una diferencia máxima de 0.0 entre los valores numéricos. Este resultado confirma que la extensión multivaluada preserva la corrección matemática del solver.

Un hallazgo relevante es que el número de iteraciones hasta convergencia difiere entre codificaciones. La codificación binaria requiere consistentemente más iteraciones que la factorizada: de 3 a 11 iteraciones para la binaria, frente a 3 a 6 para la factorizada. Esta diferencia se analiza en la sección de discusión.

En este punto se espera que ambas codificaciones produzcan funciones de valor y políticas idénticas a la solución de referencia para todos los tamaños evaluados, confirmando que la extensión preserva la corrección matemática del solver. El número de iteraciones hasta convergencia debería ser el mismo bajo ambas codificaciones, dado que resuelven el mismo MDP con los mismos parámetros γ y ϵ.

5.3. Simplificación del proceso de modelado

El segundo objetivo evalúa el impacto de la extensión sobre la complejidad del programa ProbLog que el usuario debe escribir. La Tabla 5 presenta las métricas estructurales de ambas codificaciones para cada tamaño de grid.

Tabla 5: Complejidad estructural de los programas ProbLog. La codificación factorizada mantiene un tamaño constante de 54 líneas y 27 cláusulas independientemente del tamaño del grid, mientras que la codificación binaria crece de 92 a 421 líneas.

	Fluentes		**Reglas esp./excl.**		**Cláusulas**		**Líneas totales**	
Grid	Bin.	Fact.	Bin.	Fact.	Bin.	Fact.	Bin.	Fact.
2×3	3	2	15	0	19	27	92	54
3×3	4	2	30	0	55	27	133	54
4×4	2	2	49	0	61	27	165	54
5×5	5	2	103	0	196	27	260	54
6×6	6	2	266	0	335	27	421	54

Los datos revelan tres observaciones. La primera es que la codificación factorizada utiliza exactamente 2 fluentes (fila y columna) y produce un programa de 54 líneas y 27 cláusulas en todos los tamaños evaluados. La estructura del programa es independiente del tamaño del grid porque las reglas de transición se expresan en términos de coordenadas, no de combinaciones de bits.

La segunda observación es que la codificación binaria requiere un número creciente de reglas para manejar los estados y codificar las transiciones. El número de reglas de exclusión o manejo de estados espurios crece de 15 en el grid 2×3 a 266 en el grid 6×6. Este crecimiento refleja el esfuerzo de modelado que la codificación factorizada elimina por completo.

La tercera observación concierne al tamaño del espacio de estados. La Tabla 6 presenta la comparación directa.

Tabla 6: Tamaño del espacio de estados y factor de inflación. El factor de inflación mide cuántas veces más grande es el espacio binario respecto al factorizado.

Grid	**Celdas**	$\lvert\mathcal{S}\rvert_{\text{bin}}$	$\lvert\mathcal{S}\rvert_{\text{fact}}$	**Factor de inflación**
2×3	6	8	6	1.33×
3×3	9	16	9	1.78×
4×4	16	16	16	1×
5×5	25	32	25	1.28×
6×6	36	64	36	1.78×

El factor de inflación varía entre 1.28× y 1.78× dependiendo de la relación entre el número de celdas y la potencia de 2 más cercana. El grid 4×4 presenta el factor mas bajo (1×) debido a

que en este caso el tamaño del espacio de estados es una potencia de 2. Este resultado indica que la codificación binaria implementada no optimiza el número de bits al mínimo teórico.

5.4. Impacto computacional

El tercer objetivo cuantifica el efecto de la representación sobre el comportamiento computacional del solver. Se reportan el tamaño del circuito compilado y los tiempos de las fases del pipeline.

5.4.1. Tamaño del circuito compilado

El tamaño del circuito compilado refleja la complejidad de la fórmula proposicional que ProbLog debe construir para evaluar las probabilidades de transición. Dado que la codificación binaria introduce estados espurios que el compilador debe representar (incluso para descartarlos), se espera que los circuitos de la codificación binaria sean de mayor tamaño. El grid 4×4 es el punto de control de esta hipótesis: al no tener estados espurios, cualquier diferencia en el tamaño del circuito para ese grid se debe exclusivamente a la estructura de las reglas de transición, no a la presencia de combinaciones inválidas.

5.4.2. Tiempos de ejecución

La Tabla 7 presenta los tiempos de las fases del pipeline para cada configuración.

Tabla 7: Tiempos de ejecución por fase del pipeline (en segundos). Se reporta media ± desviación estándar sobre 10 repeticiones.

Grid	**Cod.**	t_{prepare} (s)	t_{VI} (s)	t_{total} (s)
2×3	Bin.	0.023 ± 0.004	0.014 ± 0.002	0.037 ± 0.005
	Fact.	0.041 ± 0.006	0.021 ± 0.004	0.063 ± 0.007
3×3	Bin.	0.049 ± 0.004	0.035 ± 0.001	0.084 ± 0.005
	Fact.	0.048 ± 0.005	0.035 ± 0.006	0.083 ± 0.008
4×4	Bin.	0.117 ± 0.005	0.128 ± 0.003	0.245 ± 0.006
	Fact.	0.063 ± 0.005	0.069 ± 0.001	0.132 ± 0.005
5×5	Bin.	0.210 ± 0.007	0.211 ± 0.006	0.421 ± 0.011
	Fact.	0.072 ± 0.003	0.130 ± 0.003	0.202 ± 0.004
6×6	Bin.	0.407 ± 0.024	0.724 ± 0.035	1.131 ± 0.047
	Fact.	0.096 ± 0.006	0.247 ± 0.006	0.343 ± 0.010

En los grids pequeños (2×3 y 3×3), los tiempos de ambas codificaciones son comparables y del orden de decenas de milisegundos. A partir del grid 4×4, la codificación factorizada comienza a mostrar tiempos menores en ambas fases del pipeline. La diferencia se amplifica con el tamaño del grid: en el grid 6×6, la codificación binaria requiere 1.131 segundos frente a 0.343 segundos de la factorizada.

La Tabla 8 cuantifica la relación de tiempos entre ambas codificaciones.

Tabla 8: Relación de tiempos entre codificaciones (*speedup*). Un valor mayor que 1.0 indica que la codificación factorizada es más rápida. Se calcula como $t_{\text{bin}}/t_{\text{fact}}$ usando las medias.

Grid	Speedup preparación	Speedup VI	Speedup total
2×3	0.57×	0.63×	0.59×
3×3	1.03×	1.00×	1.02×
4×4	1.87×	1.84×	1.85×
5×5	2.91×	1.62×	2.09×
6×6	4.25×	2.93×	3.30×

5.5. Discusión

Los resultados experimentales permiten evaluar la extensión propuesta desde los tres ejes planteados en el diseño experimental.

5.5.1. Corrección y convergencia

Ambas codificaciones producen funciones de valor idénticas para los cinco tamaños evaluados, confirmando que la extensión preserva la corrección del solver. Sin embargo, la codificación binaria requiere consistentemente más iteraciones hasta convergencia (de 3 a 11) que la factorizada (de 3 a 6). Esta diferencia se explica por el mayor tamaño del espacio de estados en la codificación binaria. Los estados espurios reciben inicialmente un valor de cero y el algoritmo de Iteración de Valor necesita más iteraciones para propagar los valores desde la celda meta hasta los estados más lejanos a través de un espacio más grande. El efecto se amplifica con el tamaño del grid: en el grid 6×6, la codificación binaria necesita casi el doble de iteraciones que la factorizada.

5.5.2. Simplificación del modelado

La codificación factorizada produce un programa de tamaño constante (54 líneas, 27 cláusulas) independientemente del tamaño del grid, mientras que la codificación binaria crece de 92 a 421 líneas. El factor que impulsa este crecimiento es el número de reglas de manejo de estados espurios, que pasa de 15 a 266 en el rango evaluado. La codificación factorizada elimina estas reglas por completo, ya que la exclusión mutua se garantiza por la semántica de las Disyunciones Anotadas.

Este resultado tiene una consecuencia práctica directa. Al escalar el dominio, la codificación binaria requiere que el modelador escriba un número creciente de reglas proporcional al tamaño del grid, mientras que la codificación factorizada no requiere cambios en la estructura del programa. El único ajuste necesario es la actualización de las Disyunciones Anotadas que definen los valores de fila y columna.

5.5.3. Comportamiento computacional

Los tiempos de ejecución revelan un patrón que no era evidente a priori: la codificación factorizada es más lenta que la binaria para el grid más pequeño (2×3, speedup de 0.59×), comparable para el grid 3×3 (speedup de 1.02×), y progresivamente más rápida a partir del grid 4×4 hasta alcanzar un speedup de 3.30× en el grid 6×6.

La inversión de rendimiento en el grid 2×3 se explica por el costo fijo de las Disyunciones Anotadas. Para grids pequeños, el overhead de construir las ADs, inyectarlas en la ClauseDB y compilar los nodos `choice` resultantes supera el ahorro obtenido por la reducción del espacio de estados. A medida que el grid crece, la reducción del espacio de estados y del número de cláusulas compensa este costo fijo, y la ventaja de la codificación factorizada se amplifica.

La fase de preparación del MDP (clasificación, inyección, grounding y compilación) es donde la diferencia es más pronunciada. En el grid 6×6, la preparación de la codificación binaria toma 0.407 segundos frente a 0.096 de la factorizada, un factor de 4.25×. Esto es consistente con el hecho de que el programa binario contiene 335 cláusulas frente a 27 de la factorizada: más cláusulas implican un grounding más costoso y un circuito compilado de mayor tamaño.

5.5.4. Relación con las predicciones del Marco Teórico

En la Sección 3.8 se argumentó que las Disyunciones Anotadas tienen el potencial de producir circuitos más compactos al evitar la representación de combinaciones imposibles. Los datos de tiempo de preparación son consistentes con esta predicción: la fase que incluye la compilación del circuito es la que muestra la mayor diferencia entre codificaciones, lo que sugiere que el circuito generado por la codificación factorizada es efectivamente más pequeño. Una verificación directa del tamaño del circuito (número de nodos SDD o d-DNNF) confirmaría esta observación y se identifica como trabajo futuro.

5.5.5. Limitaciones de la evaluación

Los experimentos se realizaron sobre dominios de grid con transiciones deterministas y un único tipo de recompensa (celda meta). Dominios con transiciones estocásticas, múltiples fuentes de recompensa, o fluentes de estado adicionales (más allá de la posición) podrían exhibir comportamientos distintos. Laa codificación binaria utilizada en estos experimentos optimiza el número de bits al mínimo teórico $\lceil \log_2(k) \rceil$ en todos los casos, como se observó en el grid 4×4 donde se emplean 4 bits para 16 celdas.

Los resultados presentados en este capítulo proporcionan la evidencia empírica para evaluar el cumplimiento de los objetivos planteados en la Introducción. El siguiente capítulo sintetiza las contribuciones de este trabajo y plantea las líneas de investigación futura.

Índice de figuras

1. Entorno de navegación en cuadrícula utilizado para la validación de la política óptima. Adaptado de Mitchell [22]. 34

Índice de cuadros

1. Análisis comparativo del estado del arte. Se evalúan cuatro dimensiones para cada trabajo: el tipo de variables de estado soportadas, el mecanismo de inferencia, la capacidad relacional del lenguaje, y la garantía de exclusión mutua entre valores de una misma variable. 10
2. Mundos posibles y modelos mínimos inducidos por la semántica de distribución. Cada fila representa una realización total de los hechos probabilísticos independientes, su probabilidad conjunta y las consecuencias lógicas derivadas. 14
3. Tamaños de grid seleccionados para la evaluación experimental. La columna "Bits" indica el número de fluentes booleanos requeridos por la codificación binaria. Los estados espurios son combinaciones binarias que no corresponden a ninguna celda del grid. 35
4. Verificación de corrección funcional. La columna V^* indica si las funciones de valor de ambas codificaciones coinciden numéricamente (tolerancia 10^{-6}). La diferencia máxima observada fue 0.0 en todos los casos. 38
5. Complejidad estructural de los programas ProbLog. La codificación factorizada mantiene un tamaño constante de 54 líneas y 27 cláusulas independientemente del tamaño del grid, mientras que la codificación binaria crece de 92 a 421 líneas. 39
6. Tamaño del espacio de estados y factor de inflación. El factor de inflación mide cuántas veces más grande es el espacio binario respecto al factorizado. 39
7. Tiempos de ejecución por fase del pipeline (en segundos). Se reporta media ± desviación estándar sobre 10 repeticiones. 40
8. Relación de tiempos entre codificaciones (*speedup*). Un valor mayor que 1.0 indica que la codificación factorizada es más rápida. Se calcula como $t_{\text{bin}}/t_{\text{fact}}$ usando las medias. 41

Índice de algoritmos

1. Inferencia de tipo por intersección de orígenes estocásticos 26
2. Cálculo Expected Value 31

Referencias

[1] Luc De Raedt, Angelika Kimmig y Hannu Toivonen. "ProbLog: A probabilistic prolog and its application in link discovery". En: *IJCAI*. Vol. 7. 2007, págs. 2462-2467.

[2] Taisuke Sato. "A statistical learning method for logic programs with distribution semantics". En: *Proceedings of the 12th International Conference on Logic Programming (ICLP'95)*. Citeseer, 1995.

[3] Daan Fierens et al. "Inference and learning in probabilistic logic programs using weighted Boolean formulas". En: *Theory and Practice of Logic Programming* 15.3 (2015), págs. 358-401.

[4] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[5] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[6] Thiago P Bueno et al. "Markov decision processes specified by probabilistic logic programming: Representation and solution". En: *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*. IEEE. 2016, págs. 337-342.

[7] Joost Vennekens, Sofie Verbaeten y Maurice Bruynooghe. "Logic programs with annotated disjunctions". En: *International Conference on Logic Programming*. Springer, 2004, págs. 431-445.

[8] Craig Boutilier, Richard Dearden y Moisés Goldszmidt. "Stochastic dynamic programming with factored representations". En: *Artificial Intelligence* 121.1-2 (2000), págs. 49-107.

[9] Jesse Hoey et al. "SPUDD: Stochastic Planning using Decision Diagrams". En: *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI-99)*. 1999, págs. 279-288.

[10] Scott Sanner. "Relational Dynamic Influence Diagram Language (RDDL): Language Description". Unpublished Manuscript, Australian National University. 2010.

[11] Luc De Raedt y Angelika Kimmig. "Probabilistic (logic) programming concepts". En: *Machine Learning* 100.1 (2015), págs. 5-47.

[12] Guy Van den Broeck et al. "DTProbLog: A Decision-Theoretic Probabilistic Prolog". En: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*. AAAI Press, 2010, págs. 1217-1222.

[13] Adnan Darwiche y Pierre Marquis. "A knowledge compilation map". En: *Journal of Artificial Intelligence Research* 17 (2002), págs. 229-264.

[14] Adnan Darwiche. "SDD: A New Canonical Representation of Propositional Knowledge Bases". En: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, págs. 819-826.

[15] Jonas Vlasselaer et al. "ProbLog2: Inference and learning in a probabilistic logic programming language". En: *Theory and Practice of Logic Programming* 16.5-6 (2016), págs. 673-689.

[16] Craig Boutilier, Thomas Dean y Steve Hanks. "Decision-theoretic planning: Structural assumptions and computational leverage". En: *Journal of Artificial Intelligence Research* 11 (1999), págs. 1-94.

[17] Davide Nitti, Vaishak Belle y Luc De Raedt. "Planning in discrete and continuous Markov decision processes by probabilistic programming". En: *Machine Learning* 104.2-3 (2016), págs. 327-342.

[18] Fabrizio Riguzzi y Terrance Swift. "Well-definedness and efficient inference for probabilistic logic programming under the distribution semantics". En: *Theory and Practice of Logic Programming* 13.2 (2013), págs. 279-302.

[19] Joost Vennekens, Marc Denecker y Maurice Bruynooghe. "CP-logic: A language of causal probabilistic events and its relation to logic programming". En: *Theory and Practice of Logic Programming* 9.3 (2009), págs. 245-308.

[20] David Poole. "The Independent Choice Logic for Modelling Multiple Agents under Uncertainty". En: *Artificial Intelligence* 94.1-2 (1997), págs. 7-56.

[21] Richard S Sutton y Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[22] Tom M. Mitchell. *Machine Learning*. New York: McGraw-Hill, 1997.